第 4 章　常用组合逻辑电路

- 编码器
- 译码器
- 数据选择器
- 二进制加法器
- 数值比较器

演示文稿 Presentation

4.1 编码器

主要要求：

- 理解编码器的概念。
- 理解常用编码器的类型、逻辑功能和使用方法。

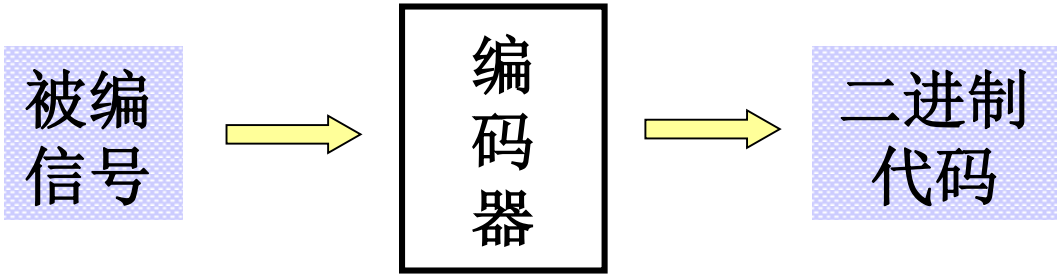
演示文稿 Presentation



一、编码器的概念与类型

编码　将具有特定含义的信息编成相应二进制代码的过程。

编码器（即Encoder）　实现编码功能的电路



编码器
- 二进制编码器
- 二-十进制编码器
- 优先编码器

二进制编码器

将一系列信号状态编制成二进制代码。

*n*个二进制代码（*n*位二进制数）有2^n种不同的组合，可以表示2^n个信号。

- 三位二进制编码器

  逻辑图

  逻辑表达式

  真值表

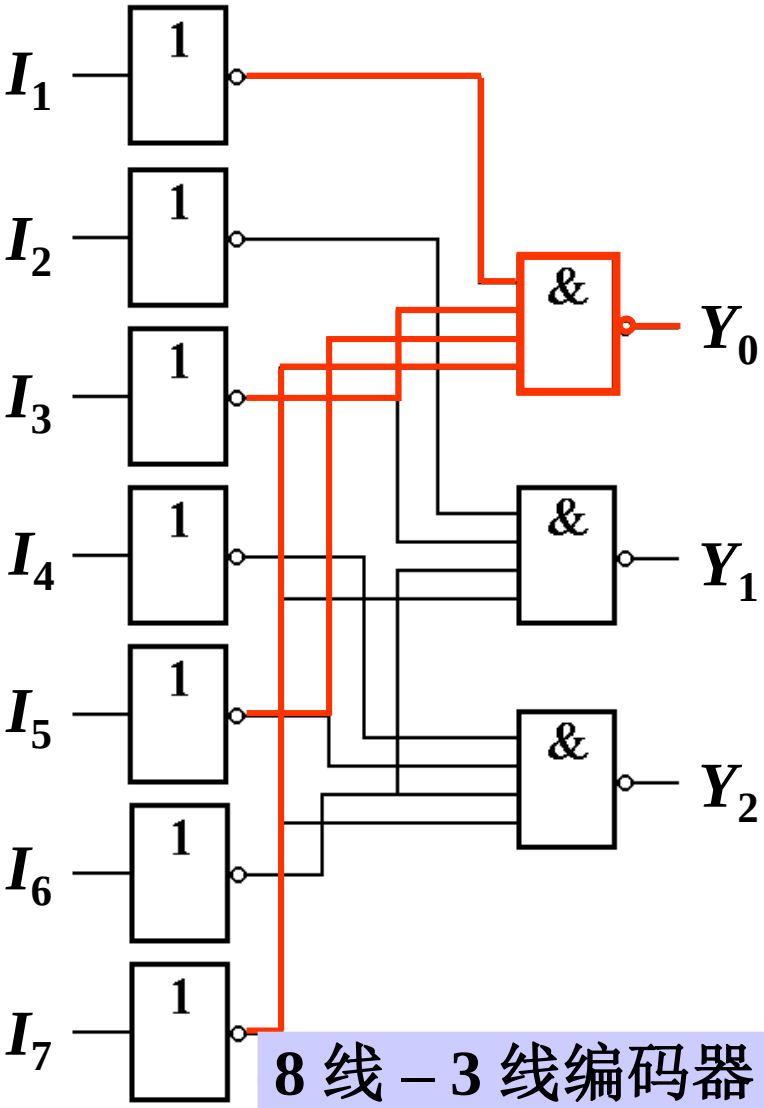
例：用与非门组成三位二进制编码器

--- 八线 - 三线编码器

设八个输入端为I_0~I_7，八种状态，与之对应的输出设为Y_0、Y_1、Y_2，共三位二进制数。

设计编码器的过程与设计一般的组合逻辑电路相同，首先要列出状态表，然后写出逻辑表达式并进行化简，最后画出逻辑图。

二、二进制编码器



8 线 – 3 线编码器

$$Y_0=\overline{\overline{I_1}\cdot\overline{I_3}\cdot\overline{I_5}\cdot\overline{I_7}}$$

$$Y_1=\overline{\overline{I_2}\cdot\overline{I_3}\cdot\overline{I_6}\cdot\overline{I_7}}$$

$$Y_2=\overline{\overline{I_4}\cdot\overline{I_5}\cdot\overline{I_6}\cdot\overline{I_7}}$$

输入								输出		
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Y_2	Y_1	Y_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

原码输出

被编信号高电平有效。

演示文稿 Presentation

三、二—十进制编码器

将十个状态（对应于十进制的十个代码）编制成BCD码。

十个输入 → 需要几位输出？ → 四位

输入：I_0~ I_9。

输出：F_3 ~ F_0

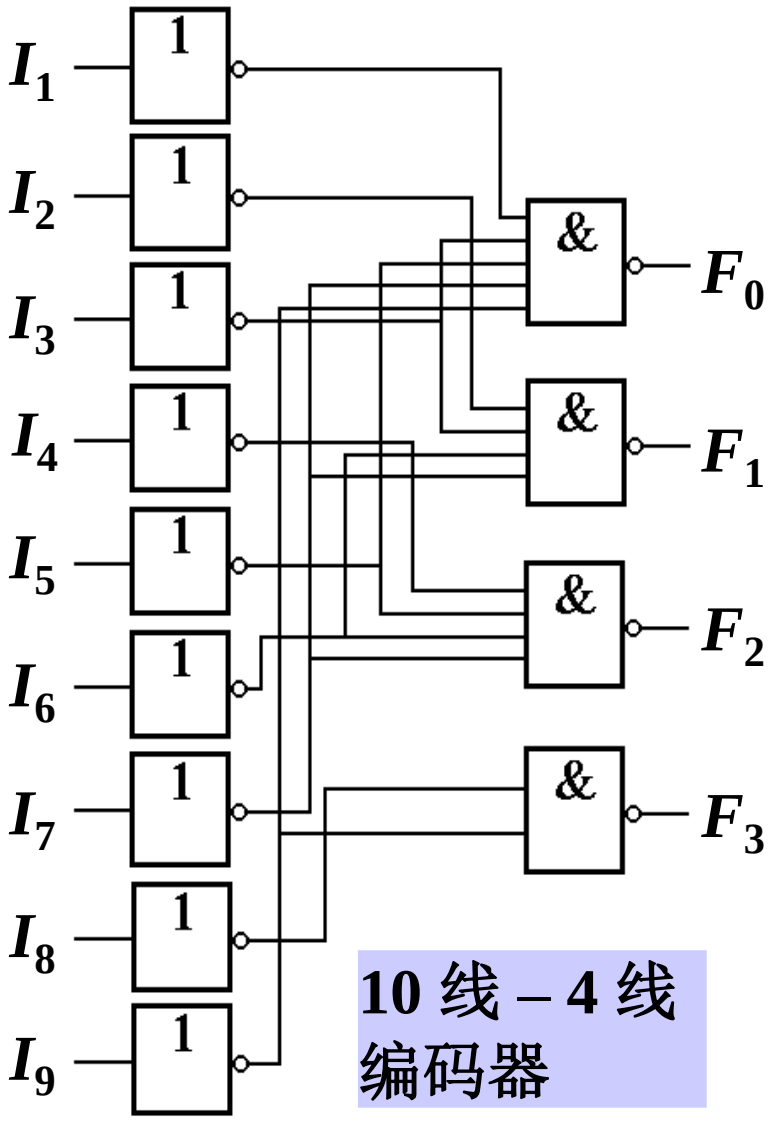
列出状态表如下：

输入	F_3	F_2	F_1	F_0
I_0	0	0	0	0
I_1	0	0	0	1
I_2	0	0	1	0
I_3	0	0	1	1
I_4	0	1	0	0
I_5	0	1	0	1
I_6	0	1	1	0
I_7	0	1	1	1
I_8	1	0	0	0
I_9	1	0	0	1

$$F_3 = \overline{I}_8 + \overline{I}_9 = \overline{I_8 \bullet I_9} \qquad F_2 = \overline{I_4 I_5 I_6 I_7}$$

$$F_1 = \overline{I_2 I_3 I_6 I_7} \qquad F_0 = \overline{I_1 I_3 I_5 I_7 I_9}$$

三、二—十进制编码器



10 线 – 4 线编码器

输入										输出			
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	I_8	I_9	F_3	F_2	F_1	F_0
1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	1	1	1
0	0	0	0	0	0	0	0	1	0	1	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	1

被编信号高电平有效

原码输出

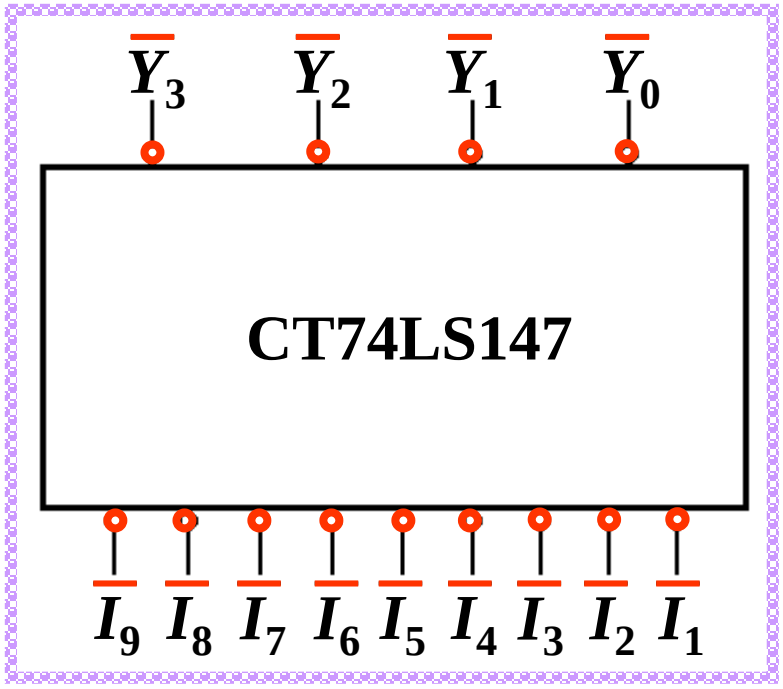
四、优先编码器（即 Priority Encoder）

允许同时输入数个编码信号，并只对其中优先权最高的信号进行编码输出的电路,，而不理睬级别低的信号。

普通编码器在任何时刻只允许一个输入端请求编码，否则输出发生混乱。

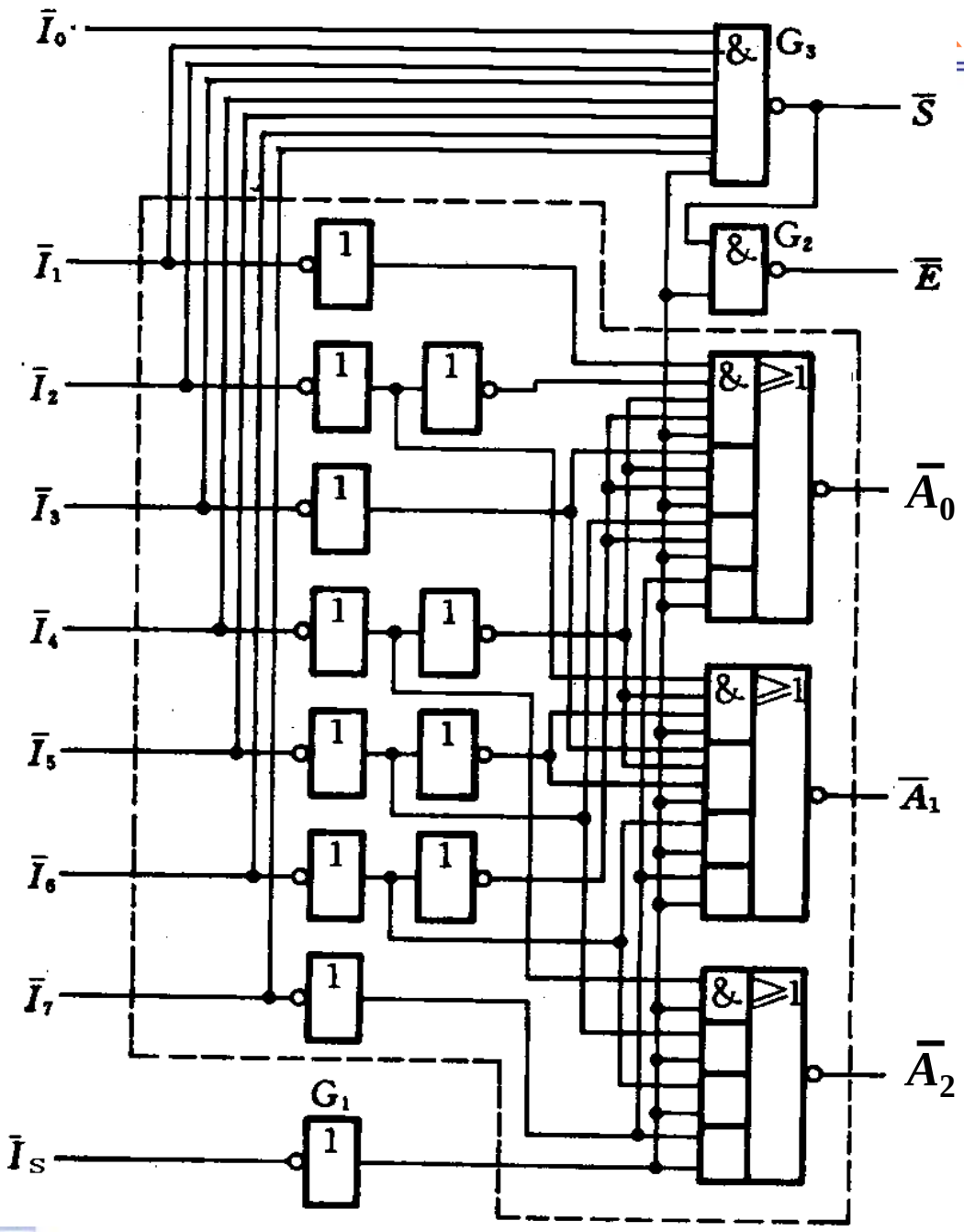
输入								输出		
I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7	Y_2	Y_1	Y_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

二－十进制优先编码器 CT74LS147



输入									输出			
$\overline{I}_1$	$\overline{I}_2$	$\overline{I}_3$	$\overline{I}_4$	$\overline{I}_5$	$\overline{I}_6$	$\overline{I}_7$	$\overline{I}_8$	$\overline{I}_9$	$\overline{Y}_3$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$
1	1	1	1	1	1	1	1	1	1	1	1	1
×	×	×	×	×	×	×	×	0	0	1	1	0
×	×	×	×	×	×	×	0	1	0	1	1	1
×	×	×	×	×	×	0	1	1	1	0	0	0
×	×	×	×	×	0	1	1	1	1	0	0	1
×	×	×	×	0	1	1	1	1	1	0	1	0
×	×	×	0	1	1	1	1	1	1	0	1	1
×	×	0	1	1	1	1	1	1	1	1	0	0
×	0	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0

被编信号优先级别从高到低依次为 $\overline{I}_9$、$\overline{I}_8$、$\overline{I}_7$、$\overline{I}_6$、$\overline{I}_5$、$\overline{I}_4$、$\overline{I}_3$、$\overline{I}_2$、$\overline{I}_1$、$\overline{I}_0$。



$\overline{I_0} - \overline{I_7}$ 为编码输入端

$\overline{I_7}$ 的优先权最高

$\overline{I_0}$ 的优先权最低

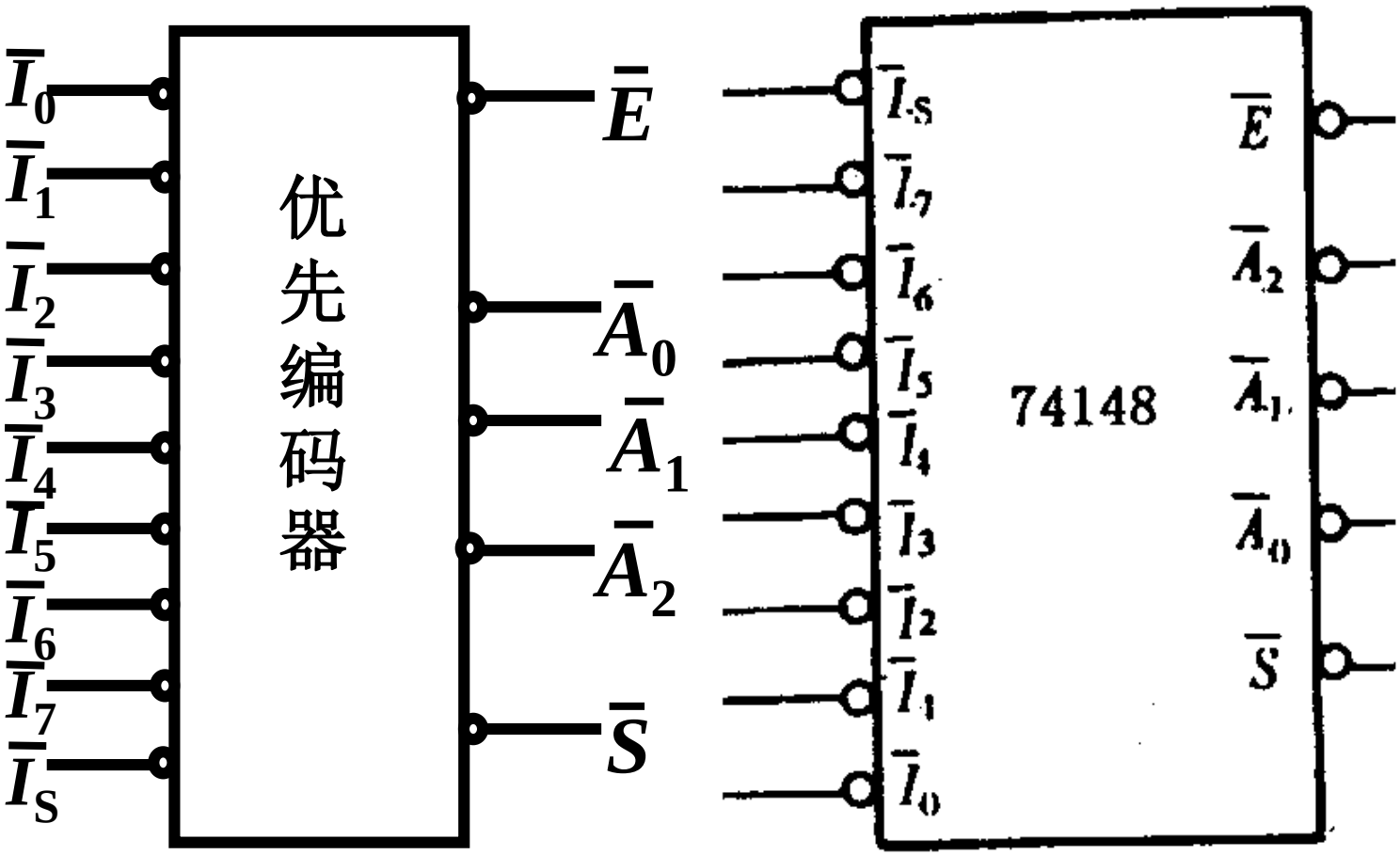
$\overline{A_2}$、$\overline{A_1}$、$\overline{A_0}$ 为编码输出端

$\overline{I_S}$ 为控制端(选通控制端)

$\overline{S}$ 为选通输出端

$\overline{E}$ 为扩展端

$\overline{S}$、$\overline{E}$ 是为扩展编码功能而设置的


$\overline{I}_0$
$\overline{I}_1$
$\overline{I}_2$
$\overline{I}_3$
$\overline{I}_4$
$\overline{I}_5$
$\overline{I}_6$
$\overline{I}_7$
$\overline{I}_S$
优先编码器
$\overline{E}$
$\overline{A}_0$
$\overline{A}_1$
$\overline{A}_2$
$\overline{S}$
74148

74148的逻辑符号

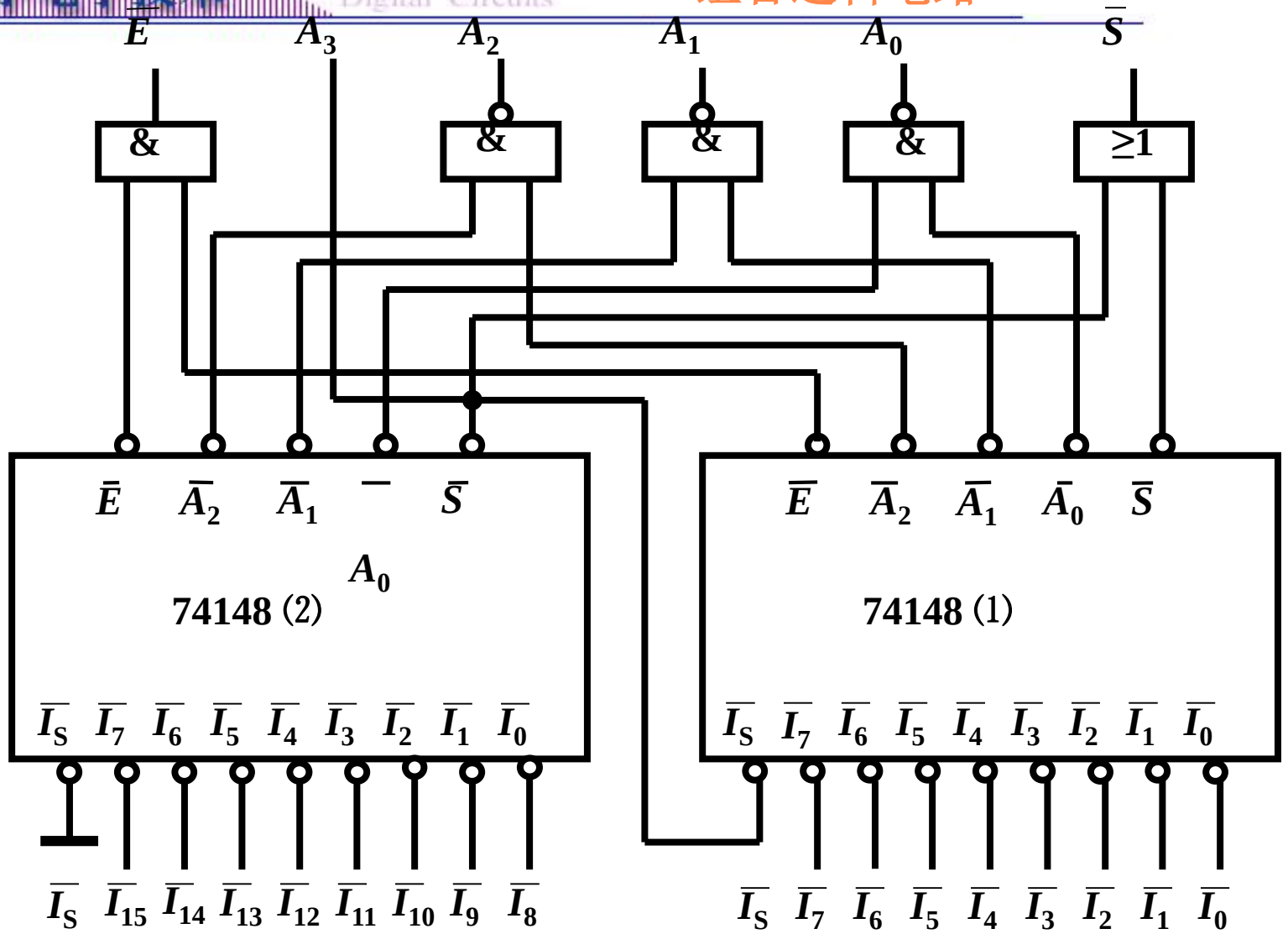


图3-14　16线-4线优先编码器

4.2 译码器

主要要求：

- 理解译码的概念。
- 掌握二进制译码器 CT74LS138 的逻辑功能和使用方法。
- 理解其他常用译码器的逻辑功能和使用方法。
- 掌握用二进制译码器实现组合逻辑电路的方法。

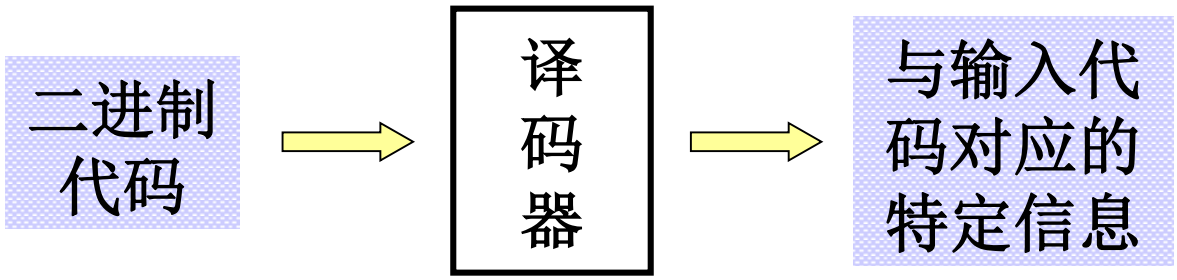
一、译码的概念与类型

译码是编码的逆过程。

将表示特定意义信息的二进制代码翻译出来。

译码器（即 Decoder）

实现译码功能的电路

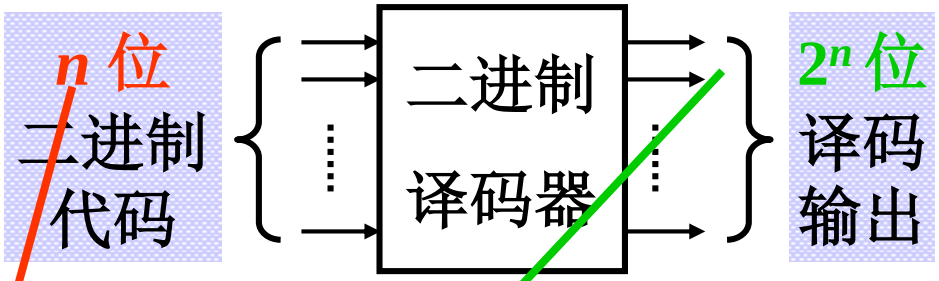


译码器

- 二进制译码器
- 二－十进制译码器
- 数码显示译码器

二、二进制译码器

将输入二进制代码译成相应输出信号的电路。



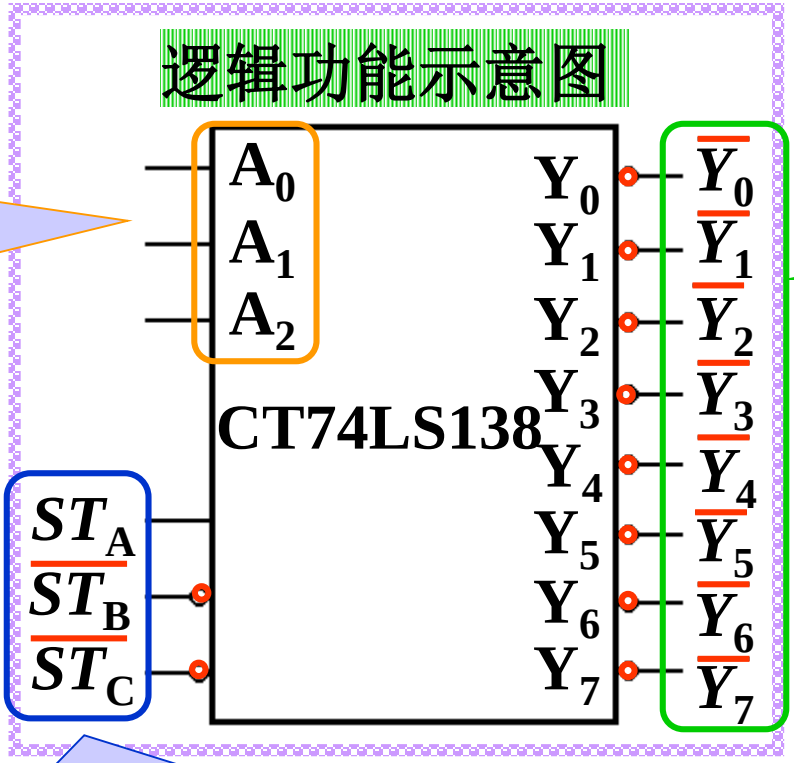
译码输入		译	码	输	出
A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

译码输出高电平有效

译码输入		译	码	输	出
A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

译码输出低电平有效

（一）3 线 － 8 线译码器 CT74LS138 简介



实物图片



使能端 ST_A 高电平有效，$\overline{ST_B}$、$\overline{ST_C}$ 低电平有效，即当 $ST_A = 1$，$\overline{ST_B} = \overline{ST_C} = 0$ 时译码，否则禁止译码。

输出逻辑函数式

$$\begin{cases}
\overline{Y}_0=\overline{\overline{A}_2\overline{A}_1\overline{A}_0}=\overline{m}_0\\
\overline{Y}_1=\overline{\overline{A}_2\overline{A}_1A_0}=\overline{m}_1\\
\overline{Y}_2=\overline{\overline{A}_2A_1\overline{A}_0}=\overline{m}_2\\
\overline{Y}_3=\overline{\overline{A}_2A_1A_0}=\overline{m}_3\\
\overline{Y}_4=\overline{A_2\overline{A}_1\overline{A}_0}=\overline{m}_4\\
\overline{Y}_5=\overline{A_2\overline{A}_1A_0}=\overline{m}_5\\
\overline{Y}_6=\overline{A_2A_1\overline{A}_0}=\overline{m}_6\\
\overline{Y}_7=\overline{A_2A_1A_0}=\overline{m}_7
\end{cases}$$

CT74LS138 真值表

输入					输出							
ST_A	$\overline{ST}_B+\overline{ST}_C$		A_2	A_1	A_0	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
×	1		×	×	×	1	1	1	1	1	1	1	1
0	×		×	×	×	1	1	1	1	1	1	1	1
1	0		0	0	0	0	1	1	1	1	1	1	1
1	0		0	0	1	1	0	1	1	1	1	1	1
1	0		0	1	0	1	1	0	1	1	1	1	1
1	0		0	1	1	1	1	1	0	1	1	1	1
1	0		1	0	0	1	1	1	1	0	1	1	1
1	0		1	0	1	1	1	1	1	1	0	1	1
1	0		1	1	0	1	1	1	1	1	1	0	1
1	0		1	1	1	1	1	1	1	1	1	1	0

二进制译码器能译出输入变量的全部取值组合，故又称变量译码器，也称全译码器。其输出端能提供输入变量的全部最小项。

（二）用二进制译码器实现组合逻辑函数

由于二进制译码器的输出端能提供输入变量的全部最小项，而任何组合逻辑函数都可以变换为最小项之和的标准式，因此用二进制译码器和门电路可实现任何组合逻辑函数。当译码器输出低电平有效时，多选用与非门；译码器输出高电平有效时，多选用或门。

演示文稿 Presentation



☼［例］试用译码器和门电路实现逻辑函数

$$Y = \overline{A}\,\overline{B}C + A\overline{B}\,\overline{C} + C$$

解：(1) 根据逻辑函数选择译码器

选用 3 线－8 线译码器 CT74LS138，
并令 $A_2 = A$，$A_1 = B$，$A_0 = C$。

(2) 将函数式变换为标准与－或式

$$\begin{aligned} Y &= \overline{A}\,\overline{B}C + A\overline{B}\,\overline{C} + C \\ &= \overline{A}\,\overline{B}C + \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC \\ &= m_1 + m_3 + m_5 + m_6 + m_7 \end{aligned}$$

(3) 根据译码器的输出有效电平确定需用的门电路

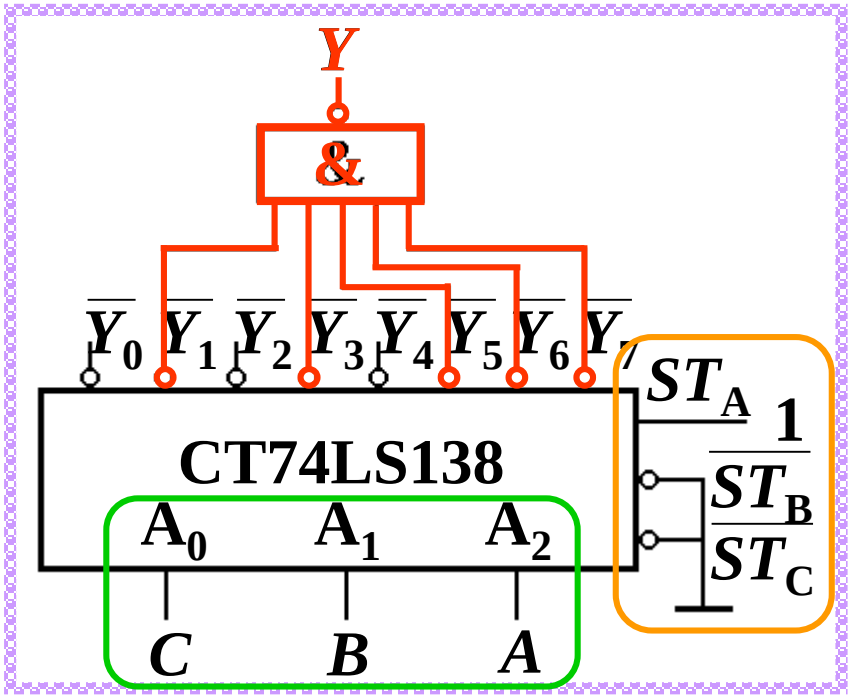
CT74LS138 输出低电平有效，$\overline{Y_i} = \overline{m_i}$，$i = 0 \sim 7$

因此，将 Y 函数式变换为 $Y = \overline{m_1 \cdot m_3 \cdot m_5 \cdot m_6 \cdot m_7}$

$$= \overline{\overline{Y_1} \cdot \overline{Y_3} \cdot \overline{Y_5} \cdot \overline{Y_6} \cdot \overline{Y_7}}$$

采用 5 输入与非门，其输入取自 $\overline{Y_1}$、$\overline{Y_3}$、$\overline{Y_5}$、$\overline{Y_6}$和 $\overline{Y_7}$。

(4) 画连线图



$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_iB_i$$

☼ ［例］试用译码器实现全加器。

解：(1) 分析设计要求，列出真值表

设被加数为 A_i，加数为 B_i，低位进位数为 C_{i-1}。
输出本位和为 S_i，向高位的进位数为 C_i。

列出全加器的真值表如下：

输入			输出	
A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(2) 根据真值表写函数式

$$\begin{cases} S_i = m_1 + m_2 + m_4 + m_7 \\ C_i = m_3 + m_5 + m_6 + m_7 \end{cases}$$

(3) 选择译码器

选用 3 线 – 8 线译码器 CT74LS138。并令 $A_2 = A_i$，$A_1 = B_i$，$A_0 = C_{i-1}$。

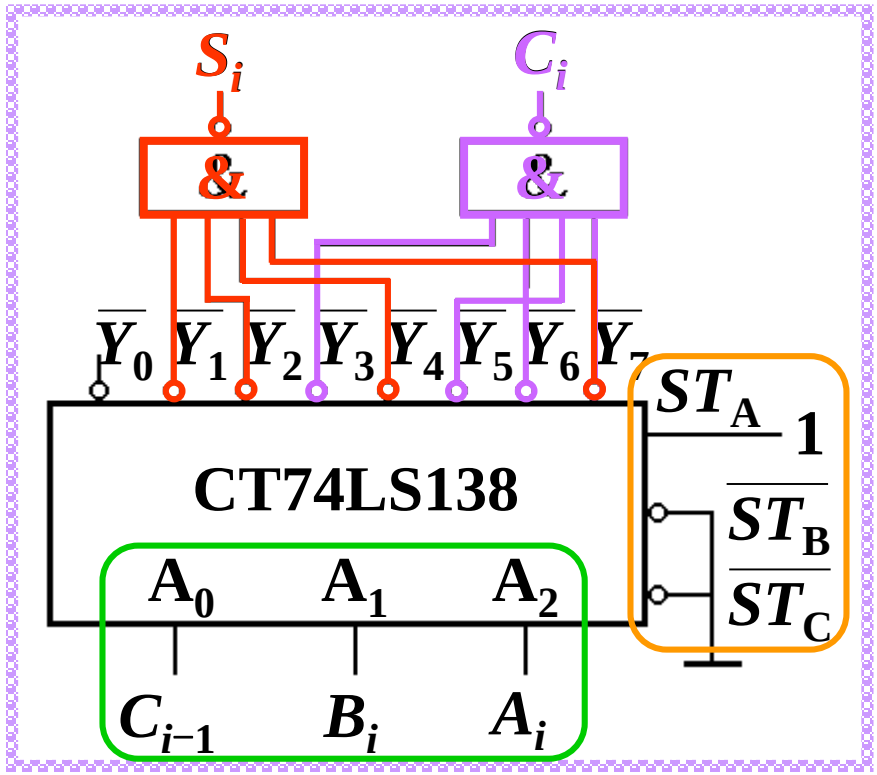
(4) 根据译码器的输出有效电平确定需用的门电路

CT74LS138 输出低电平有效，$\overline{Y_i} = \overline{m_i}$，$i = 0 \sim 7$

因此，将函数式变换为

$$\begin{cases} S_i = \overline{m_1 \cdot m_2 \cdot m_4 \cdot m_7} = \overline{\overline{Y_1} \cdot \overline{Y_2} \cdot \overline{Y_4} \cdot \overline{Y_7}} \\ C_i = \overline{m_3 \cdot m_5 \cdot m_6 \cdot m_7} = \overline{\overline{Y_3} \cdot \overline{Y_5} \cdot \overline{Y_6} \cdot \overline{Y_7}} \end{cases}$$

(5) 画连线图



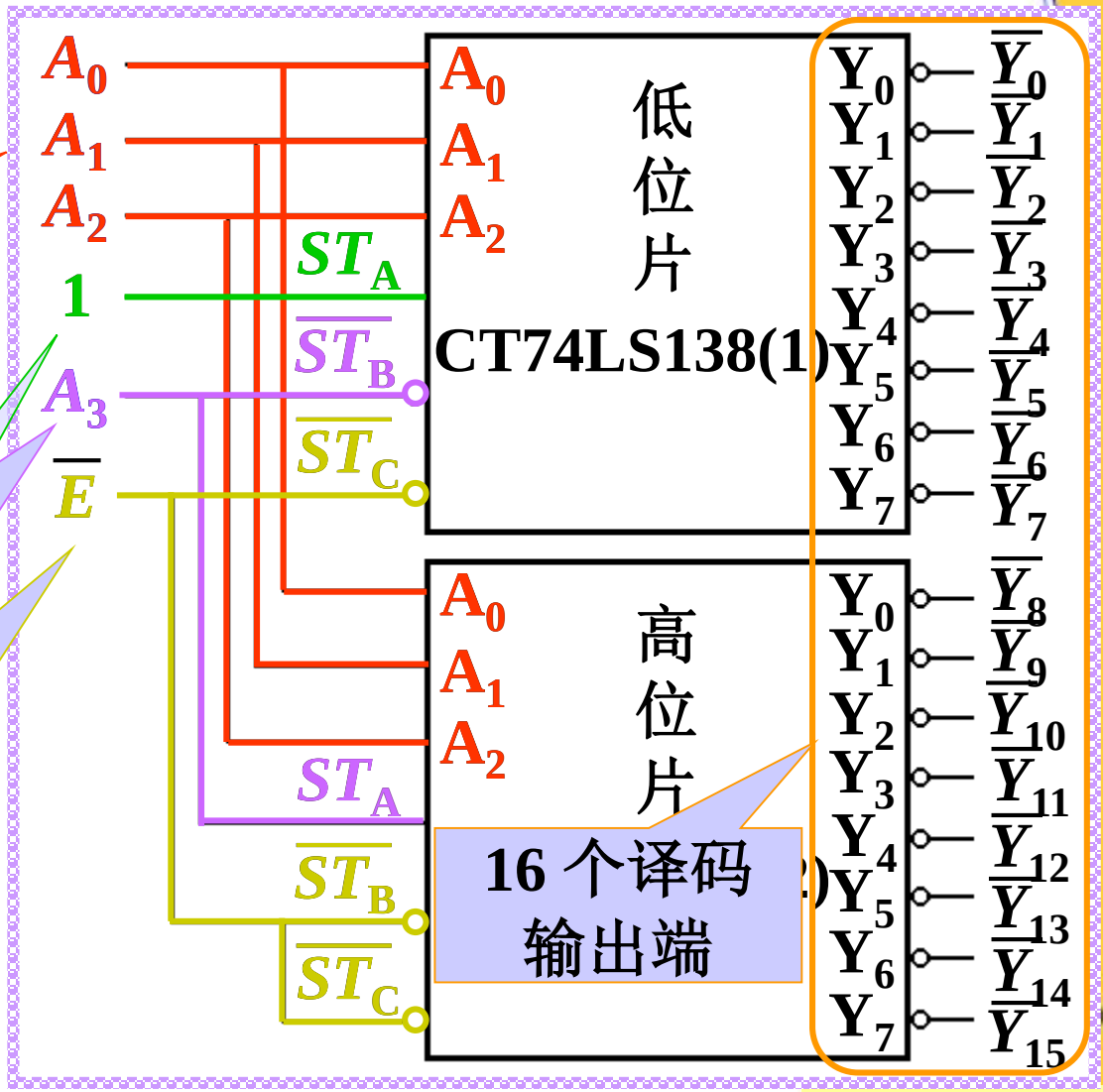
(三) 译码器的扩展

例如　两片 CT74LS138 组成的 4 线 – 16 线译码器。

低 3 位码从各译码器的码输入端输入。

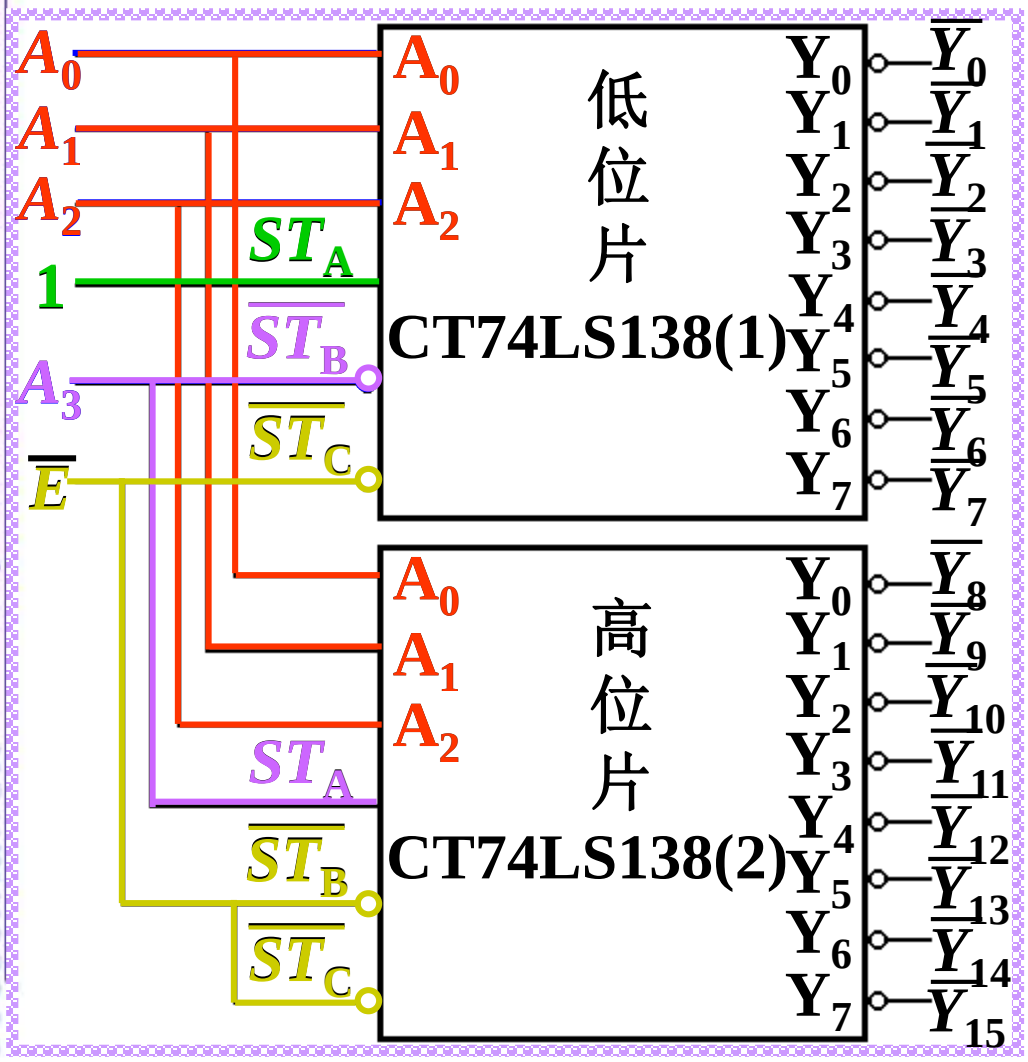
高位码 A_3 与高位片 ST_A 端和低位片 $\overline{ST}_B$ 端相连，因此，$A_3 = 0$ 时低位片工作，$A_3 = 1$ 时高位片工作。

作 4 线 – 16 线译码器使能端，低电平有效。



演示文稿 Presentation

CT74LS138 组成的 4 线 – 16 线译码器工作原理



$\overline{E}$ = 1 时，两个译码器都不工作，输出 $\overline{Y}_0 \sim \overline{Y}_{15}$ 都为高电平 1。

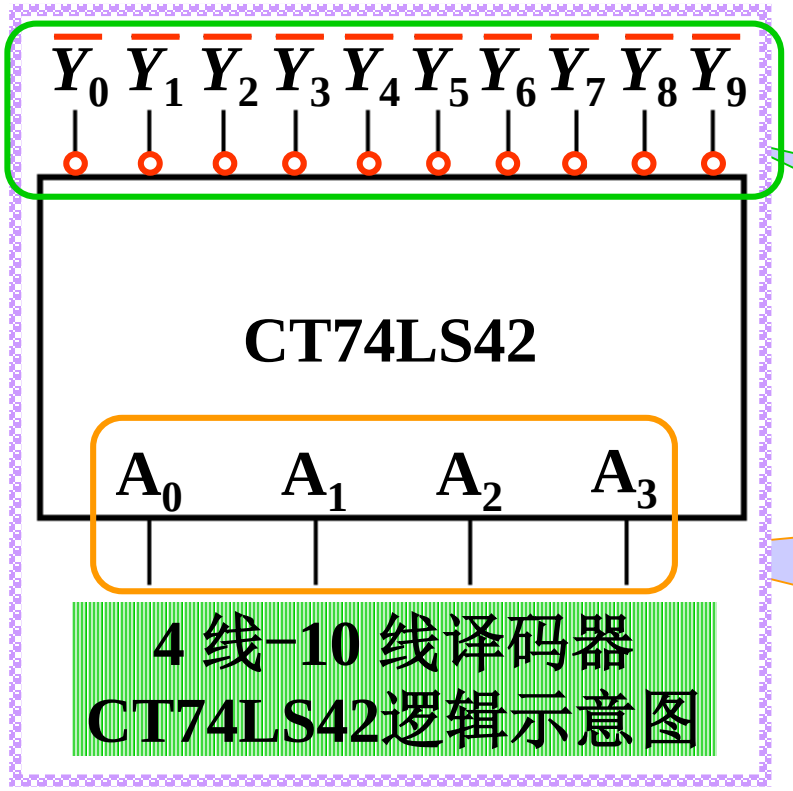
$\overline{E}$ = 0 时，允许译码。

(1) A_3 = 0 时，高位片不工作，低位片工作，译出与输入 0000 ~ 0111 分别对应的 8 个输出信号 $\overline{Y}_0 \sim \overline{Y}_7$。

(2) A_3 = 1 时，低位片不工作，高位片工作，译出与输入 1000 ~ 1111 分别对应的 8 个输出信号 $\overline{Y}_8 \sim \overline{Y}_{15}$。

三、二—十进制译码器

将 BCD 码的十组代码译成 0 ~ 9 十个对应输出信号的电路，又称 4 线 – 10 线译码器。



4 线–10 线译码器 CT74LS42逻辑示意图

10 个译码输出端，低电平 0 有效。

8421BCD 码输入端，从高位到低位依次为 A_3、A_2、A_1 和 A_0。

4 线–10 线译码器 CT74LS42 真值表

十进制数	输入 A_3	A_2	A_1	A_0	输出 $\overline{Y_0}$	$\overline{Y_1}$	$\overline{Y_2}$	$\overline{Y_3}$	$\overline{Y_4}$	$\overline{Y_5}$	$\overline{Y_6}$	$\overline{Y_7}$	$\overline{Y_8}$	$\overline{Y_9}$
0	0	0	0	0	0	1	1	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1	1	1
2	0	0	1	0	1	1	0	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	0	1	1	1	1	1	1
4	0	1	0	0	1	1	1	1	0	1	1	1	1	1
5	0	1	0	1	1	1	1	1	1	0	1	1	1	1
6	0	1	1	0	1	1	1	1	1	1	0	1	1	1
7	0	1	1	1	1	1	1	1	1	1	1	0	1	1
8	1	0	0	0	1	1	1	1	1	1	1	1	0	1
9	1	0	0	1	1	1	1	1	1	1	1	1	1	0
伪码	1	0	1	0	1	1	1	1	1	1	1	1	1	1
	1	0	1	1	1	1	1	1	1	1	1	1	1	1
	1	1	0	0	1	1	1	1	1	1	1	1	1	1
	1	1	0	1	1	1	1	1	1	1	1	1	1	1
	1	1	1	0	1	1	1	1	1	1	1	1	1	1
	1	1	1	1	1	1	1	1	1	1	1	1	1	1

四、数码显示译码器

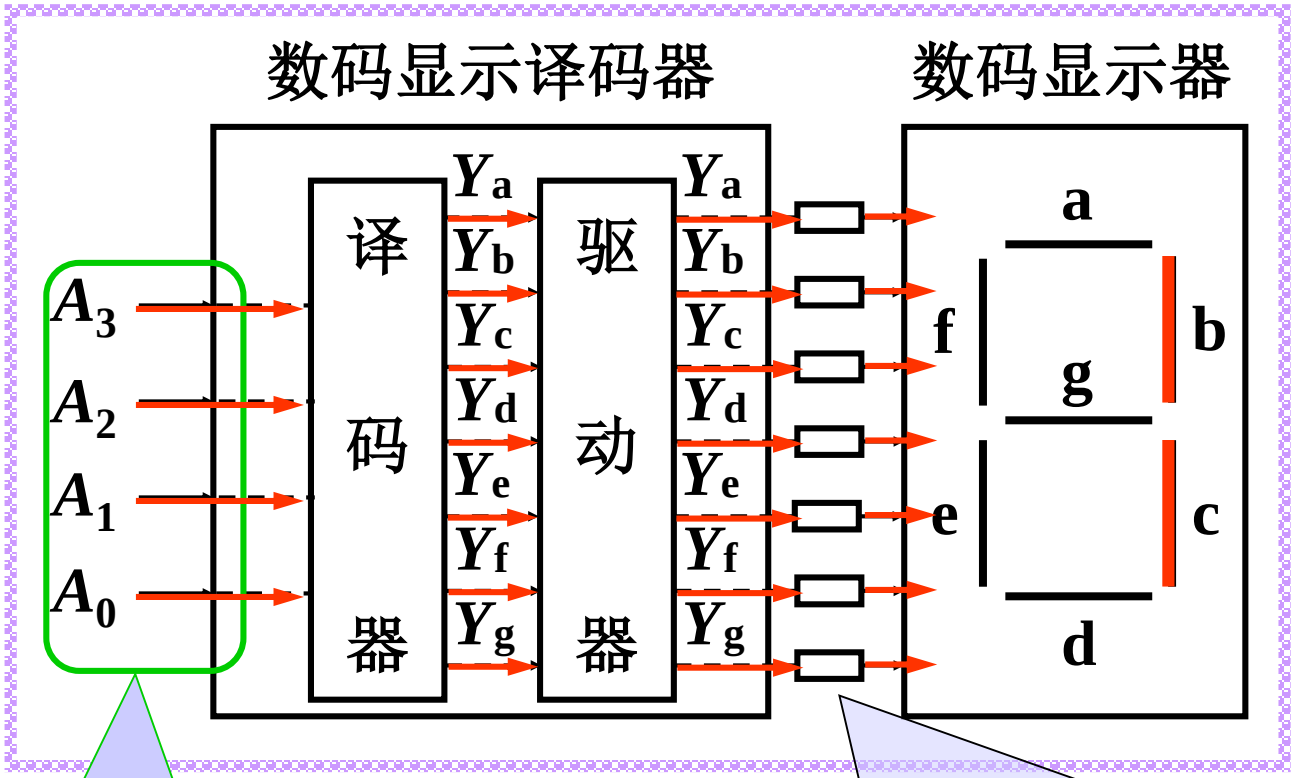
在数字系统中，常常需要将运算结果用人们习惯的十进制显示出来，这就要用到显示译码器。



四、数码显示译码器

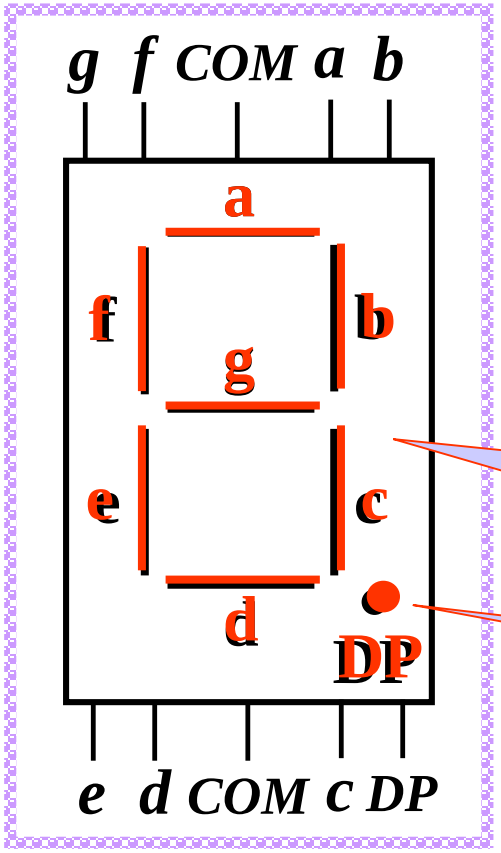
将输入的 BCD 码译成相应输出信号，以驱动显示器显示出相应数字的电路。

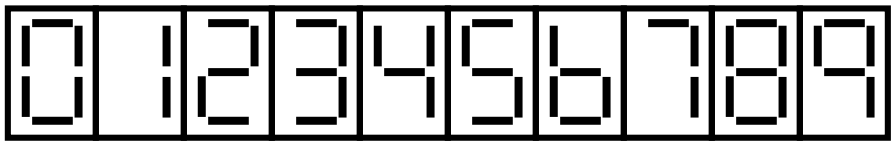
（一）数码显示译码器的结构和功能示意



(二)数码显示器简介

1. 七段半导体数码显示器(LED)





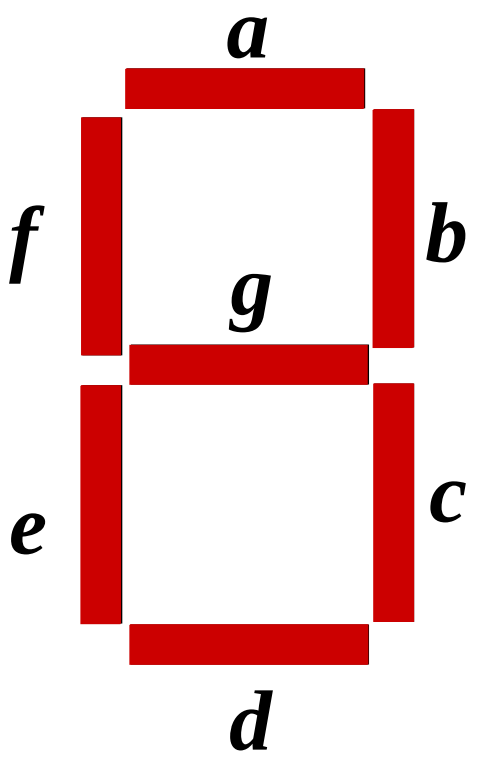
显示的数字形式

发光字段，由管脚 *a* ~ *g* 电平控制是否发光。

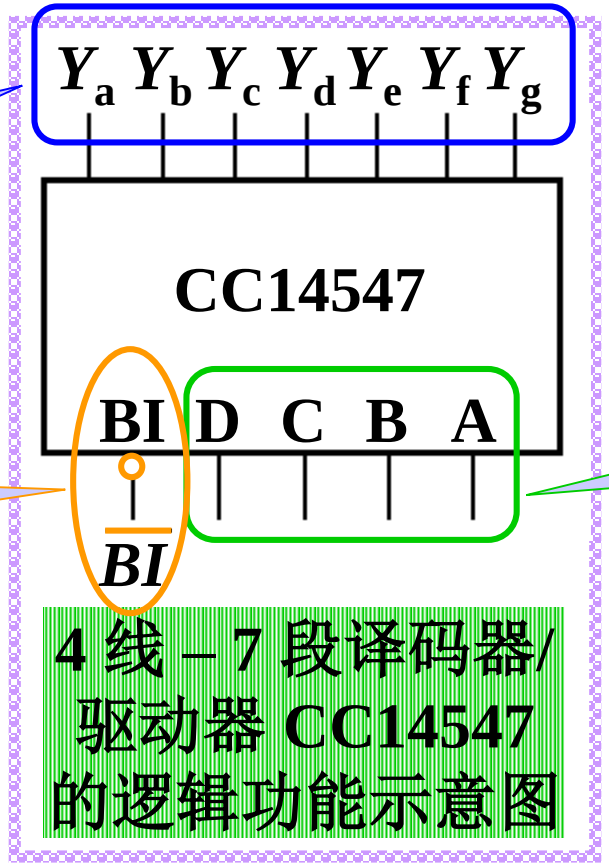
小数点，需要时才点亮。

显示器件： 常用的是七段显示器件

a	*b*	*c*	*d*	*e*	*f*	*g*
1	1	1	1	1	1	0
0	1	1	0	0	0	0
1	1	0	1	1	0	1



3. 七段显示译码器



译码驱动输出端，高电平有效。

消隐控制端，低电平有效。

8421 码输入端

4 线 – 7 段译码器/驱动器 CC14547 的逻辑功能示意图

4.3 数据选择器

主要要求：

- 理解数据选择器的作用。
- 理解常用数据选择器的逻辑功能及其使用。
- 掌握用数据选择器实现组合逻辑电路的方法。

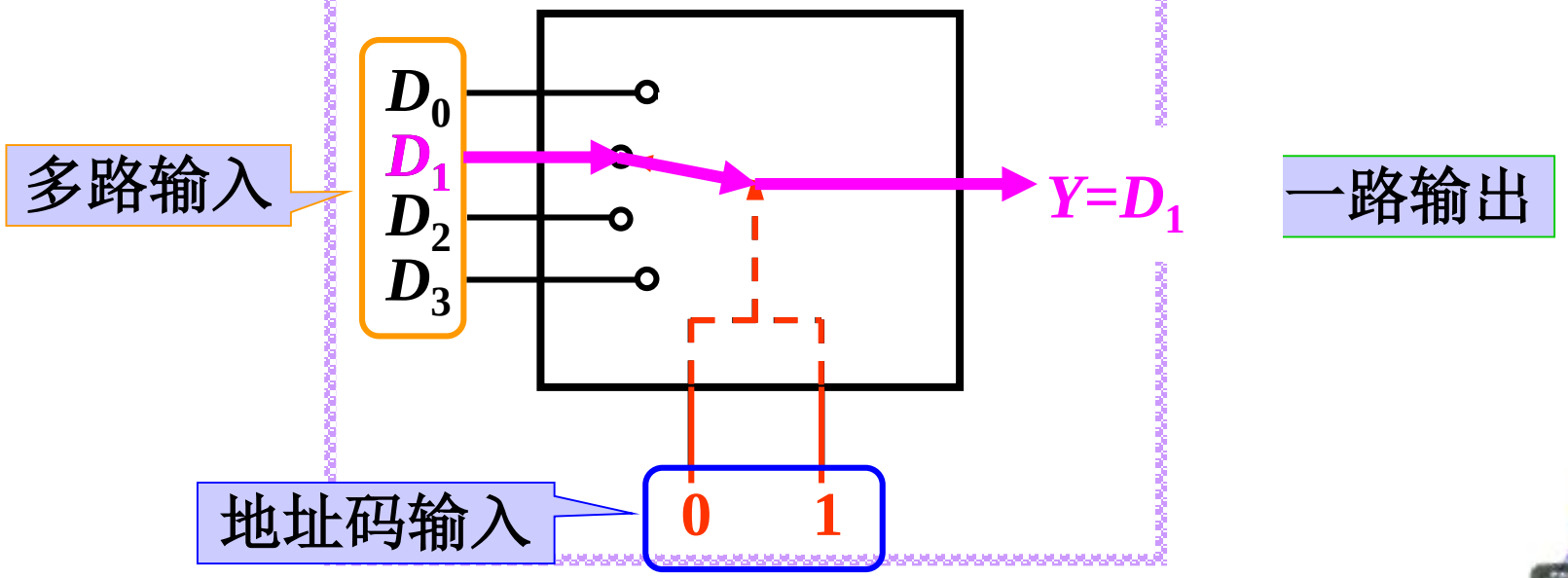
演示文稿 Presentation

一、数据选择器的作用

数据选择器： 根据地址码的要求，从多路输入信号中选择其中一路输出的电路.

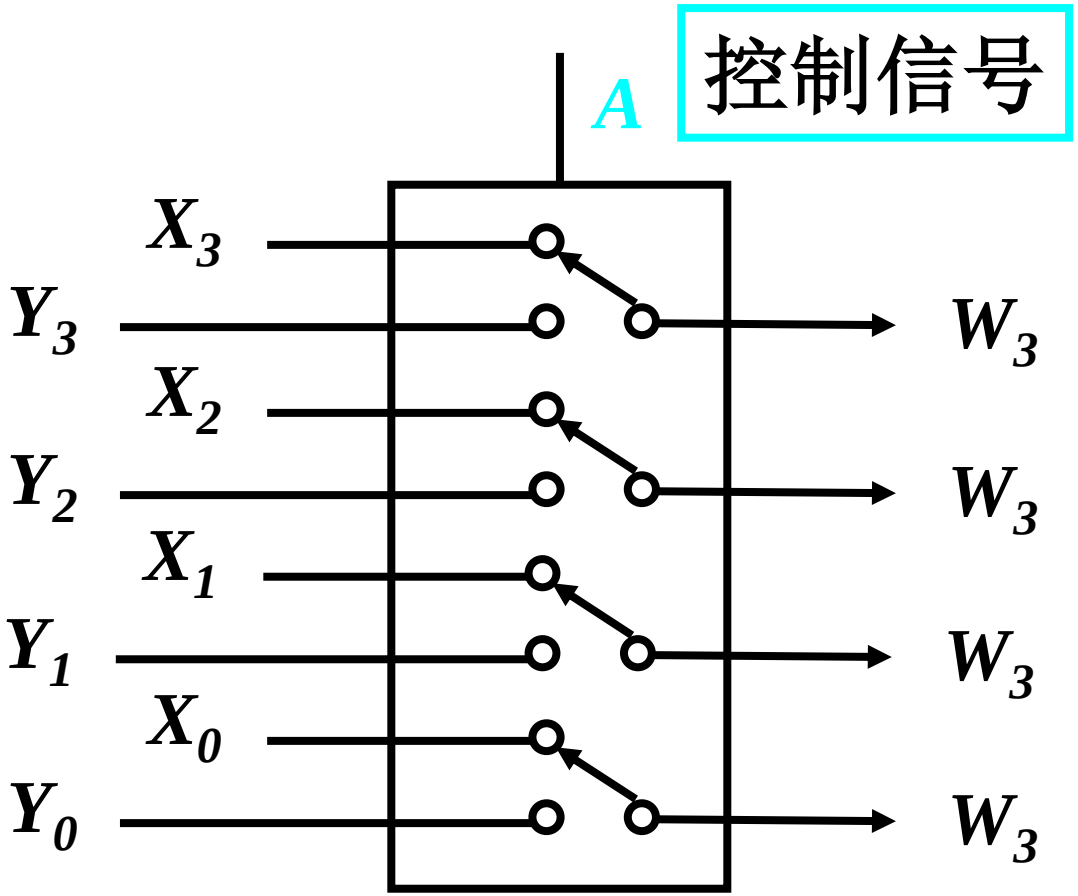
又称多路选择器(Multiplexer，简称MUX)或多路开关。

4选1数据选择器工作示意图



数据选择器的输入信号个数 N 与地址码个数 n 的关系为 $N = 2^n$

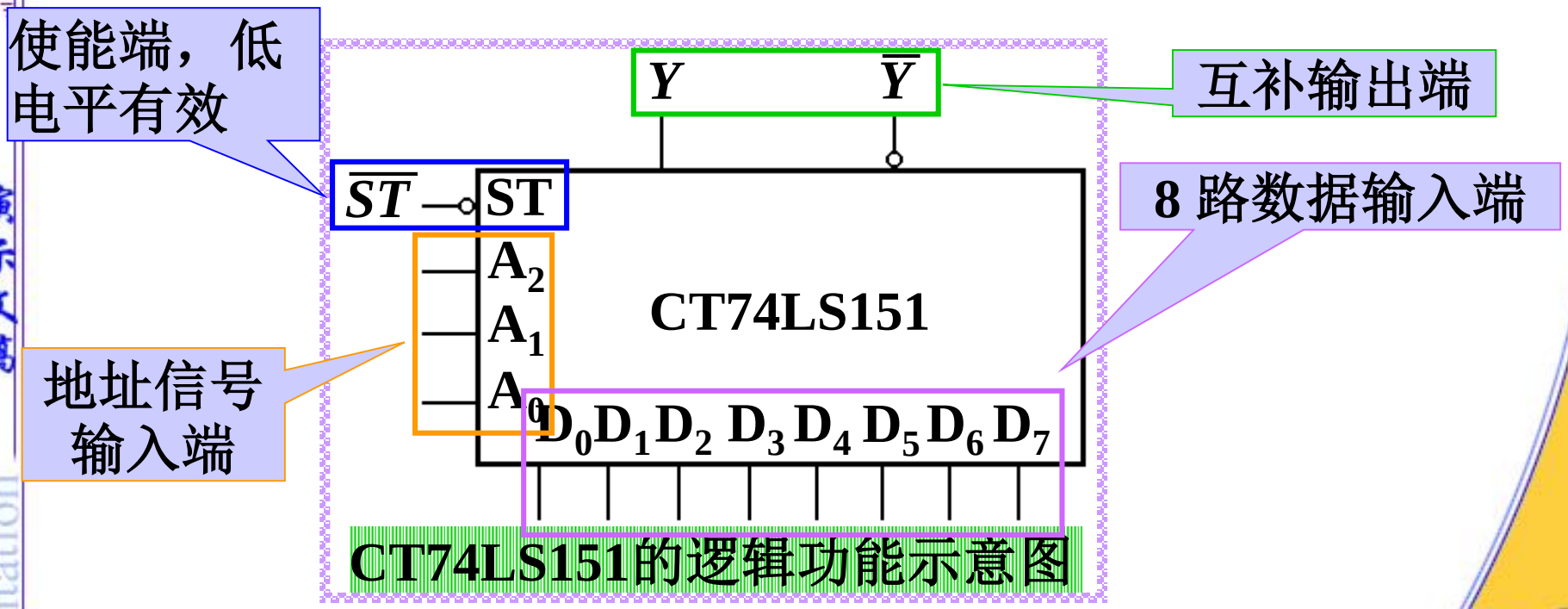
从n个数据中选择一路传输，称为**一位数据选择器**。从*m*组数据中各选择一路传输，称为***m*位数据选择器**。



四二选一选择器

二、数据选择器的逻辑功能及其使用

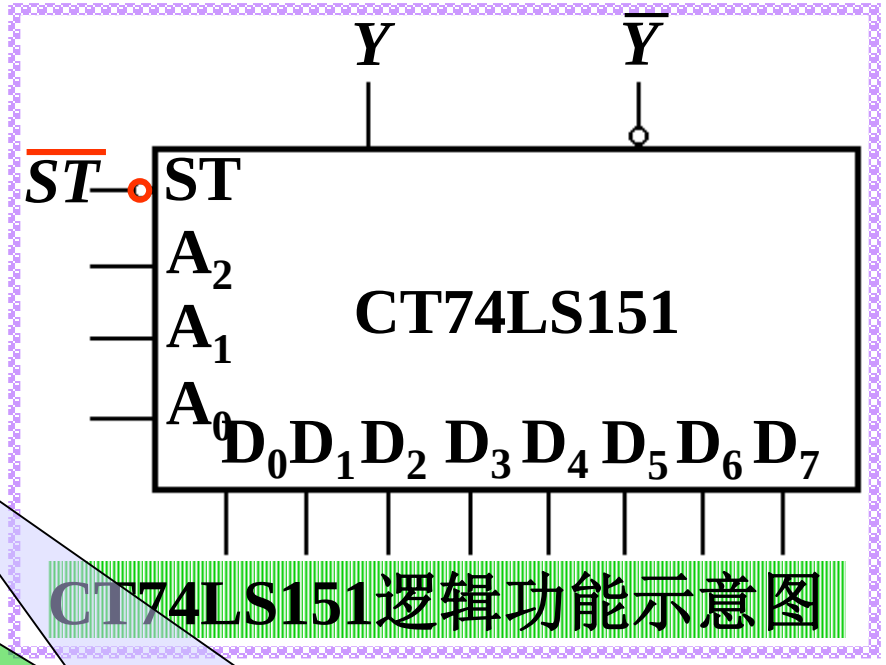
1. 8 选 1 数据选择器 CT74LS151



CT74LS151的逻辑功能示意图

8选1数据选择器 CT74LS151真值表

输入				输出	
$\overline{ST}$	A_2	A_1	A_0	Y	$\overline{Y}$
1	×	×	×	0	1
0	0	0	0	D_0	$\overline{D_0}$
0	0	0	1	D_1	$\overline{D_1}$
0	0	1	0	D_2	$\overline{D_2}$
0	0	1	1	D_3	$\overline{D_3}$
0	1	0	0	D_4	$\overline{D_4}$
0	1	0	1	D_5	$\overline{D_5}$
0	1	1	0	D_6	$\overline{D_6}$
0	1	1	1	D_7	$\overline{D_7}$



CT74LS151逻辑功能示意图

$\overline{ST}=1$ 时禁止数据选择器工作

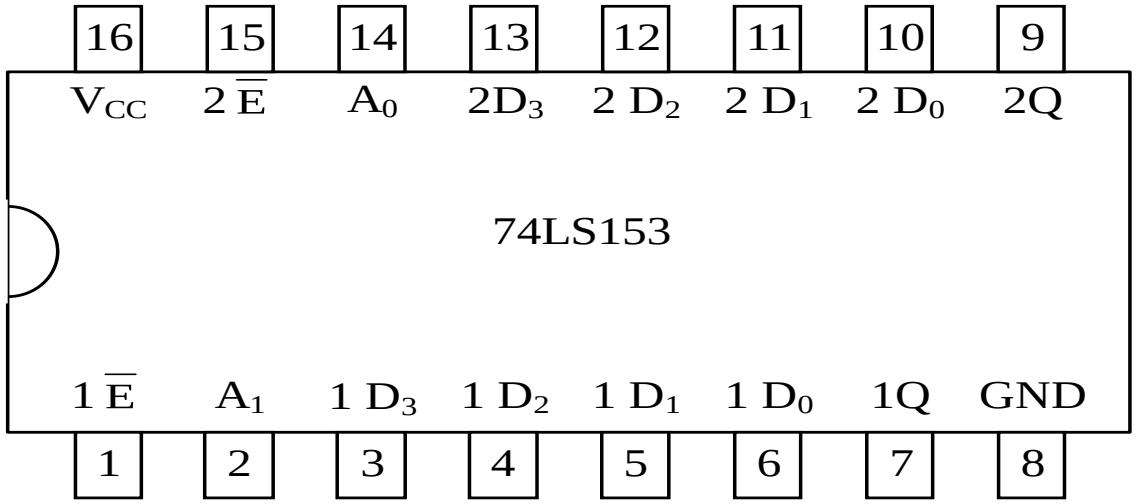
$\overline{ST}=0$ 时，数据选择器工作。选择哪一路信号输出由地址码决定。

2. 4 选 1 数据选择器 74LS153

控制端

功能表

输入			输出
A_1	A_0	$\overline{E}$	W
ϕ	ϕ	1	0
0	0	0	D_0
0	1	0	D_1
1	0	0	D_2
1	1	0	D_3



三、用数据选择器实现组合逻辑函数

由于数据选择器在输入数据全部为 1 时，输出为地址输入变量全体最小项的和。

而任何一个逻辑函数都可表示成最小项表达式，因此用数据选择器可实现任何组合逻辑函数。

当逻辑函数的变量个数和数据选择器的地址输入变量个数相同时，可直接将逻辑函数输入变量有序地接数据选择器的地址输入端。

［例］试用数据选择器实现函数 $Y = AB + AC + BC$ 。

代 数 法 求 解

解：(1) 选择数据选择器

Y为三变量函数，故选用 8 选 1 数据选择器，现选用 CT74LS151。

(2) 写出逻辑函数的最小项表达式

$$Y = AB + AC + BC = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

(3) 写出数据选择器的输出表达式

$$Y' = \overline{A_2}\,\overline{A_1}\,\overline{A_0}D_0 + \overline{A_2}\,\overline{A_1}A_0D_1 + \overline{A_2}A_1\overline{A_0}D_2 + \overline{A_2}A_1A_0D_3 + A_2\overline{A_1}\,\overline{A_0}D_4 + A_2\overline{A_1}A_0D_5 + A_2A_1\overline{A_0}D_6 + A_2A_1A_0D_7$$

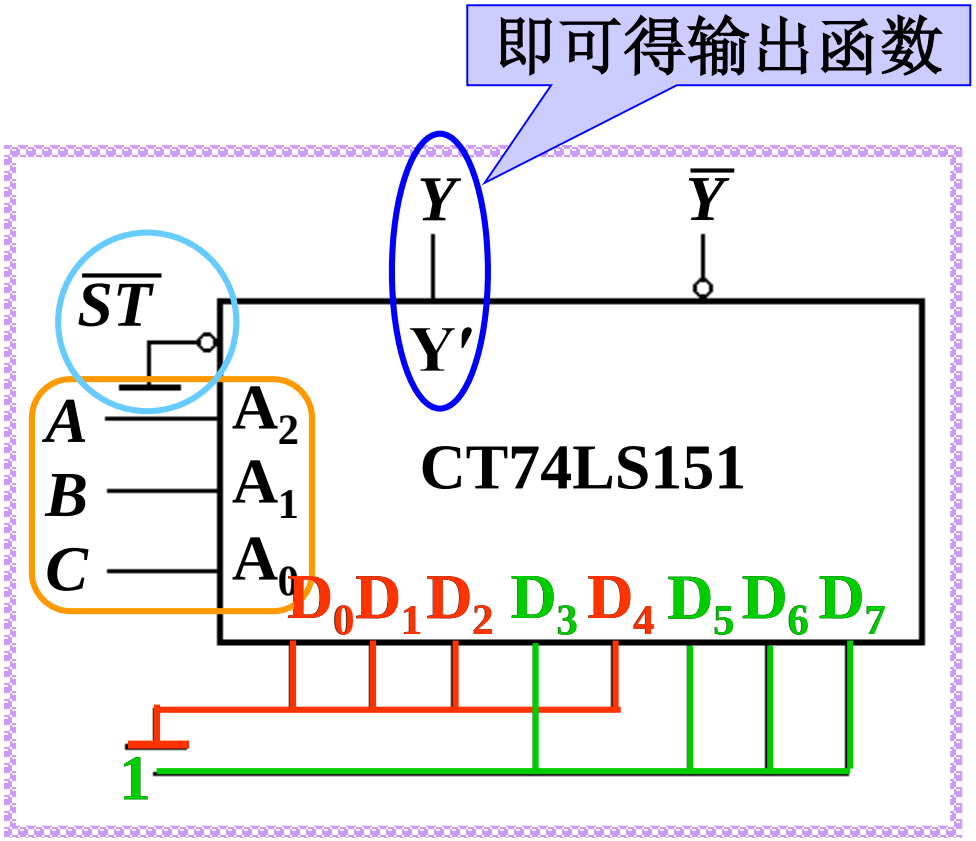
(4) 比较 Y 和 Y'两式中最小项的对应关系

令 $A = A_2$，$B = A_1$，$C = A_0$

则 $Y' = \bar{A}\bar{B}\bar{C}D_0 + \bar{A}\bar{B}CD_1 + \bar{A}B\bar{C}D_2 + \bar{A}BCD_3 + A\bar{B}\bar{C}D_4 + A\bar{B}CD_5 + AB\bar{C}D_6 + ABCD_7$

为使 $Y = Y'$，应令 $\begin{cases} D_0 = D_1 = D_2 = D_4 = 0 \\ D_3 = D_5 = D_6 = D_7 = 1 \end{cases}$

(5) 画连线图


即可得输出函数
Y
$\overline{Y}$
$\overline{ST}$
Y′
A
B
C
A_2
A_1
A_0
CT74LS151
D_0 D_1 D_2 D_3 D_4 D_5 D_6 D_7
1

卡诺图法求解

解：(1) 选择数据选择器　　选用 CT74LS151

(2) 画出 Y 和数据选择器输出 Y' 的卡诺图

Y 的卡诺图

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Y' 的卡诺图

A_2 \ A_1A_0	00	01	11	10
0	D_0	D_1	D_3	D_2
1	D_4	D_5	D_7	D_6

(3) 比较逻辑函数 Y' 和 Y 的卡诺图

设 $Y = Y'$、$A = A_2$、$B = A_1$、$C = A_0$

对比两张卡诺图后得 $\begin{cases} D_0 = D_1 = D_2 = D_4 = 0 \\ D_3 = D_5 = D_6 = D_7 = 1 \end{cases}$

(4) 画连线图　　与代数法所得图相同

4.4 二进制加法器

主要要求：

- 理解加法器的概念。
- 理解常用加法器的类型、逻辑功能和使用方法。

一、加法器

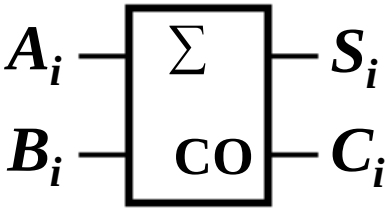
（一）加法器基本单元

半加器

Half Adder，简称 HA。它只将两个 1 位二进制数相加，而不考虑低位来的进位。

输 入		输 出	
A_i	B_i	S_i	C_i
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A\overline{B} + \overline{A}B = A \oplus B$$
$$C = AB$$



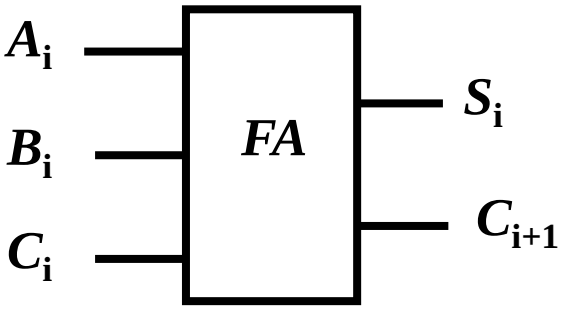
全加器

Full Adder，简称FA。能将本位的两个二进制数和邻低位来的进位数进行相加。

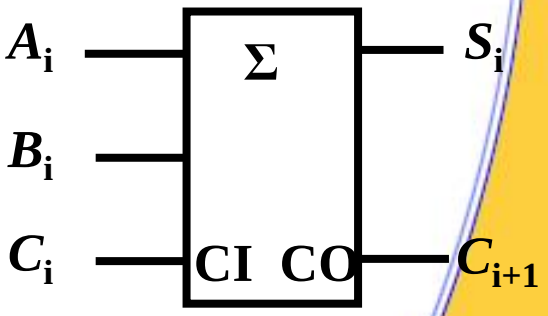
输入			输出	
A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

$$C_i = (A_i \oplus B_i)C_{i-1} + A_i B_i$$



常用逻辑符号



国标逻辑符号

加法器

- 加法器：实现多位二进制数加法运算的电路。
- 串行加法器：串行加法器采用串行运算方式，从二进制数的最低位开始，逐位相加至最高位，最后得出和数。
- 并行加法器：并行加法器采用并行运算方式，将各位数同时相加，因而提高了运算速度。

并行加法器

- 按进位数传递方式可分为串行进位和并行进位两种方式并行加法器。
- 串行进位并行加法器的全加器的个数等于相加数的位数。
- 图3-38串行进位并行加法器的逻辑图。

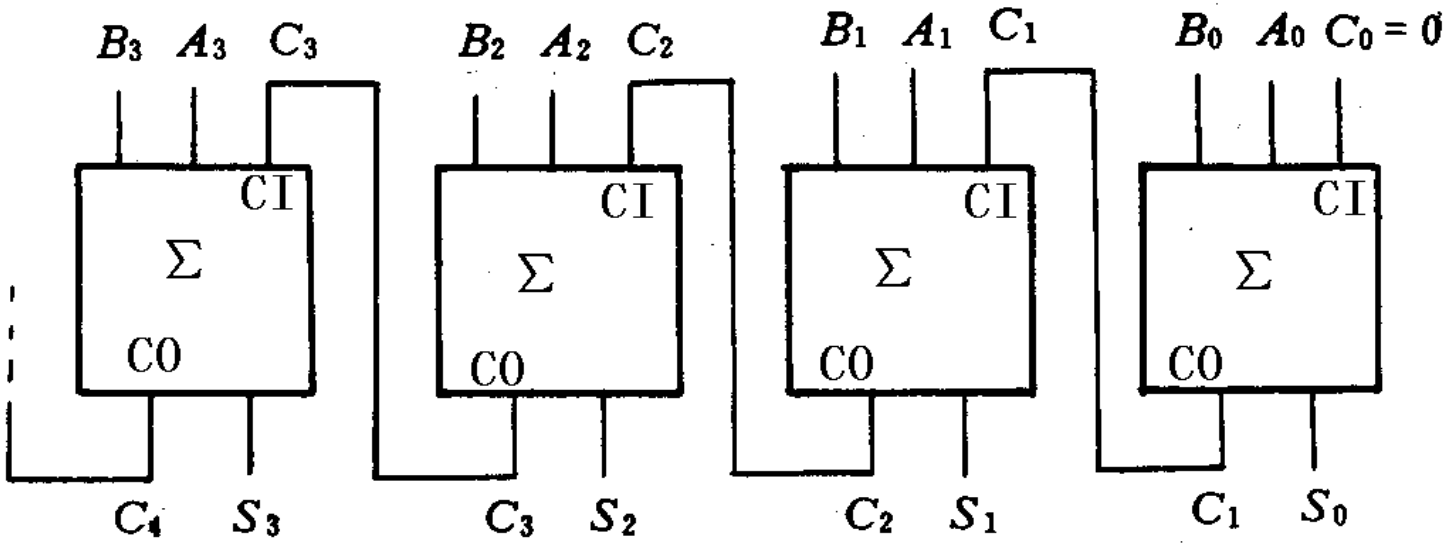
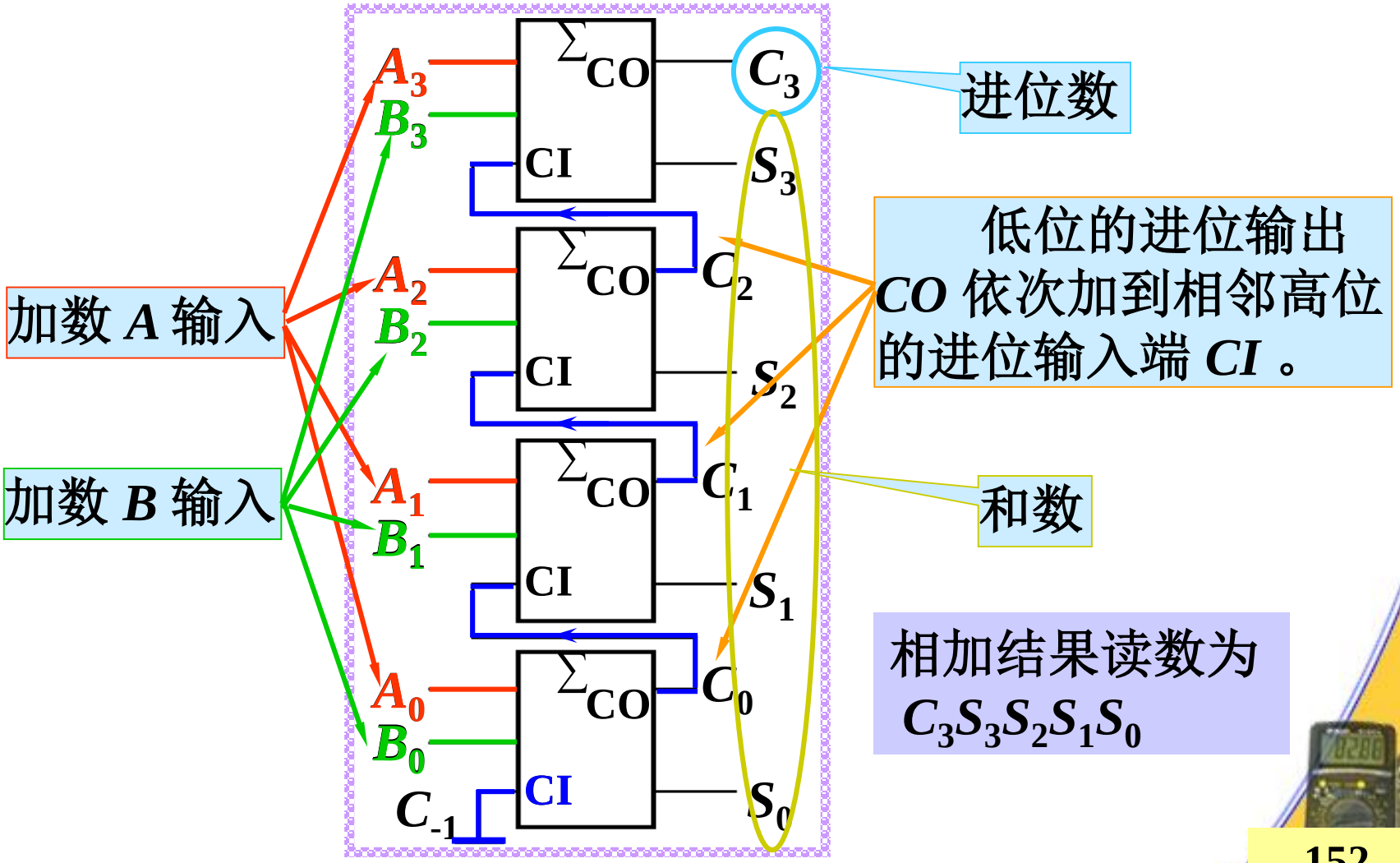


图3-38串行进位并行加法器

演示文稿 Presentation

串行进位加法器举例


加数 A 输入
加数 B 输入
A_3
B_3
A_2
B_2
A_1
B_1
A_0
B_0
C_{-1}
CO
CI
C_3
S_3
C_2
S_2
C_1
S_1
C_0
S_0
进位数
低位的进位输出 CO 依次加到相邻高位的进位输入端 CI 。
和数
相加结果读数为 $C_3S_3S_2S_1S_0$

串行进位并行加法器

- 全加器的个数等于加数的位数。
- 优点是电路简单、连接方便；
- 缺点是运算速度不高。最高位的运算，必须等到所有低位运算依次结束，送来进位信号之后才能进行。

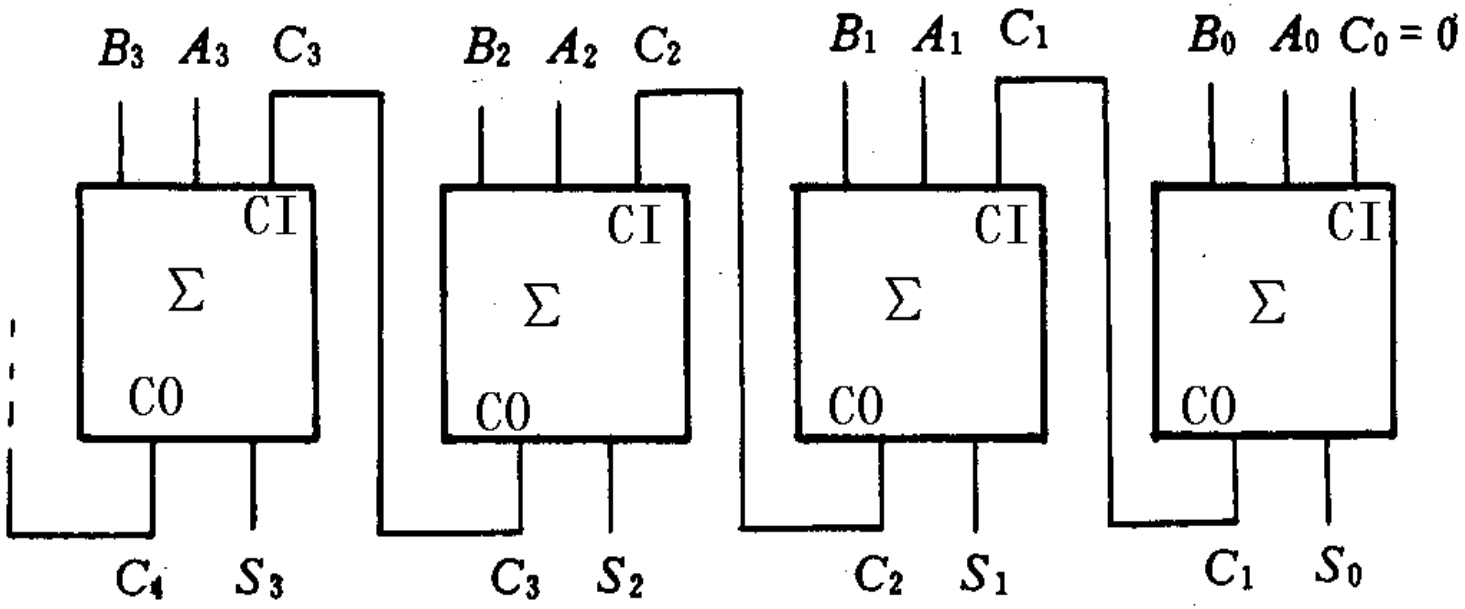


图3-38串行进位并行加法器

超前进位加法器设计思想：

每位全加器输出本位和S_i。已知全加器的进位信号的逻辑表达式为：$C_i=A_iB_i+(A_i \oplus B_i)C_{i-1}$

令$P_i=A_i \oplus B_i$,$G_i=A_iB_i$，则有$C_i=G_i+P_iC_{i-1}$

进位传递函数

进位产生函数

$$C_0 = G_0 + P_0C_{-1}$$

$$C_1 = G_1 + P_1C_0$$

$$C_2 = G_2 + P_2C_1$$

$$C_3 = G_3 + P_3C_2$$

演示文稿 Presentation

由这些表达式画出的超前进位电路实现了相对进位信号的快速传递。

各位和如下式：

$$S_0=A_0\oplus B_0\oplus C_0$$
$$S_1=A_1\oplus B_1\oplus C_1$$
$$S_2=A_2\oplus B_2\oplus C_2$$
$$S_3=A_3\oplus B_3\oplus C$$

和数信号与进位信号是同时产生的，不必逐级传送。因而，提高了运算速度。

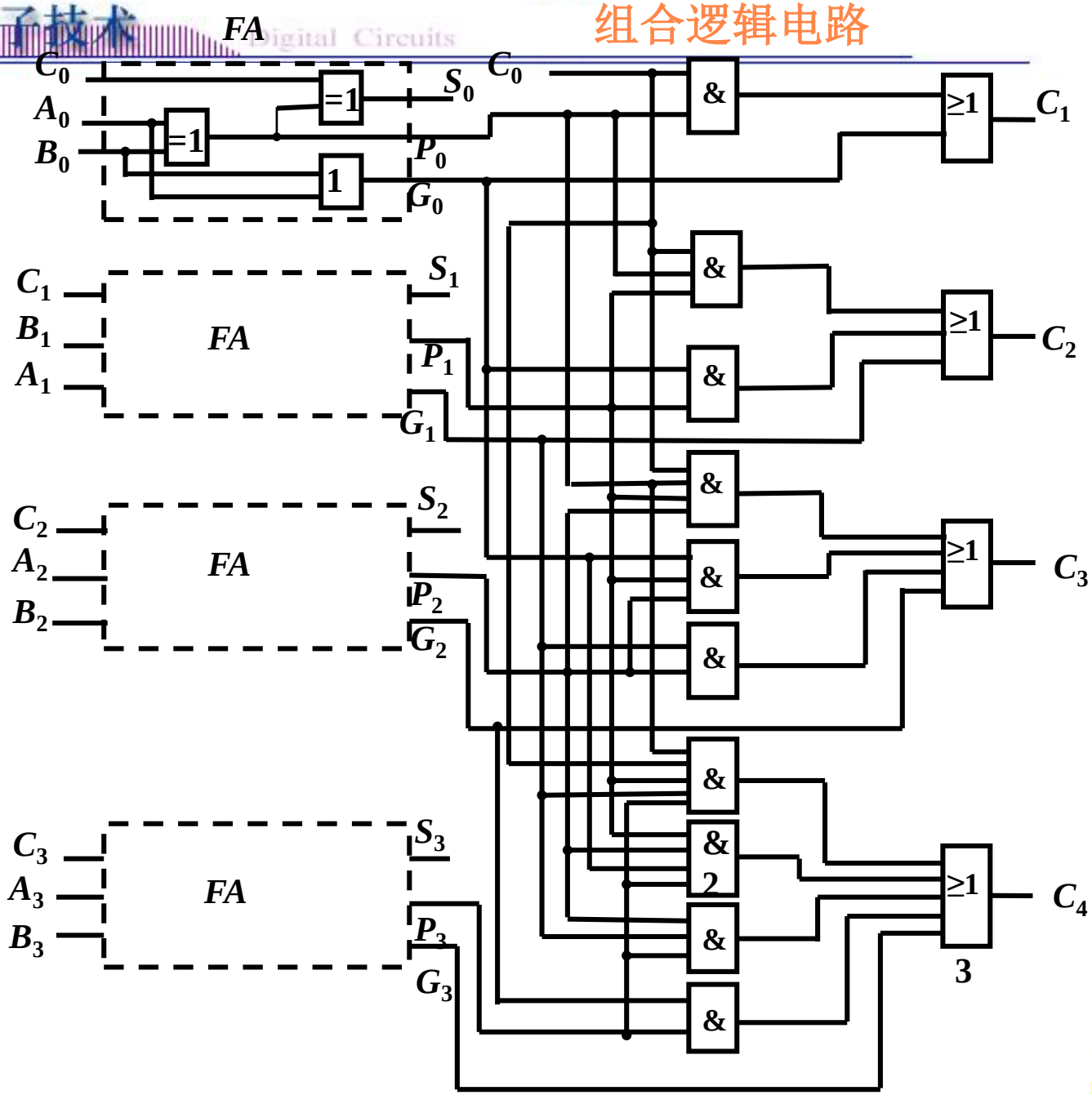
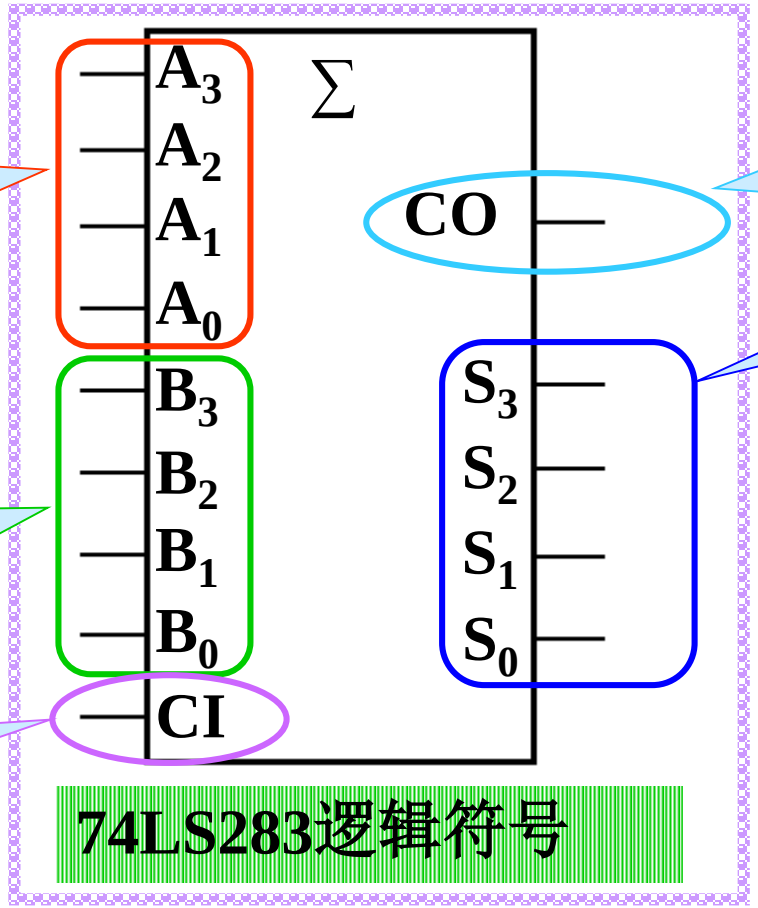


图3-39 四位超前进位并行加法器

演示文稿 Presentation

超前进位加法器举例：74LS283



多位加法器除了可以实现加法运算功能之外，还可以实现组合逻辑电路。

例：将8421BCD码转换成余3码。

余3码＝8421BCD码＋3(即0011)

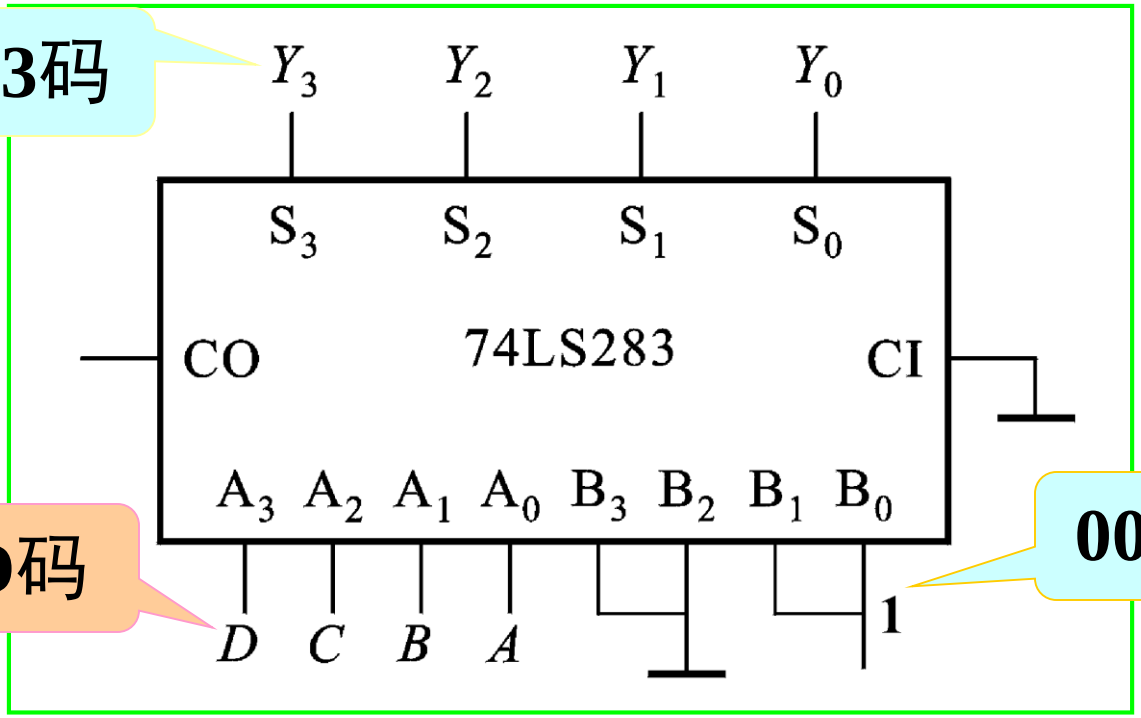
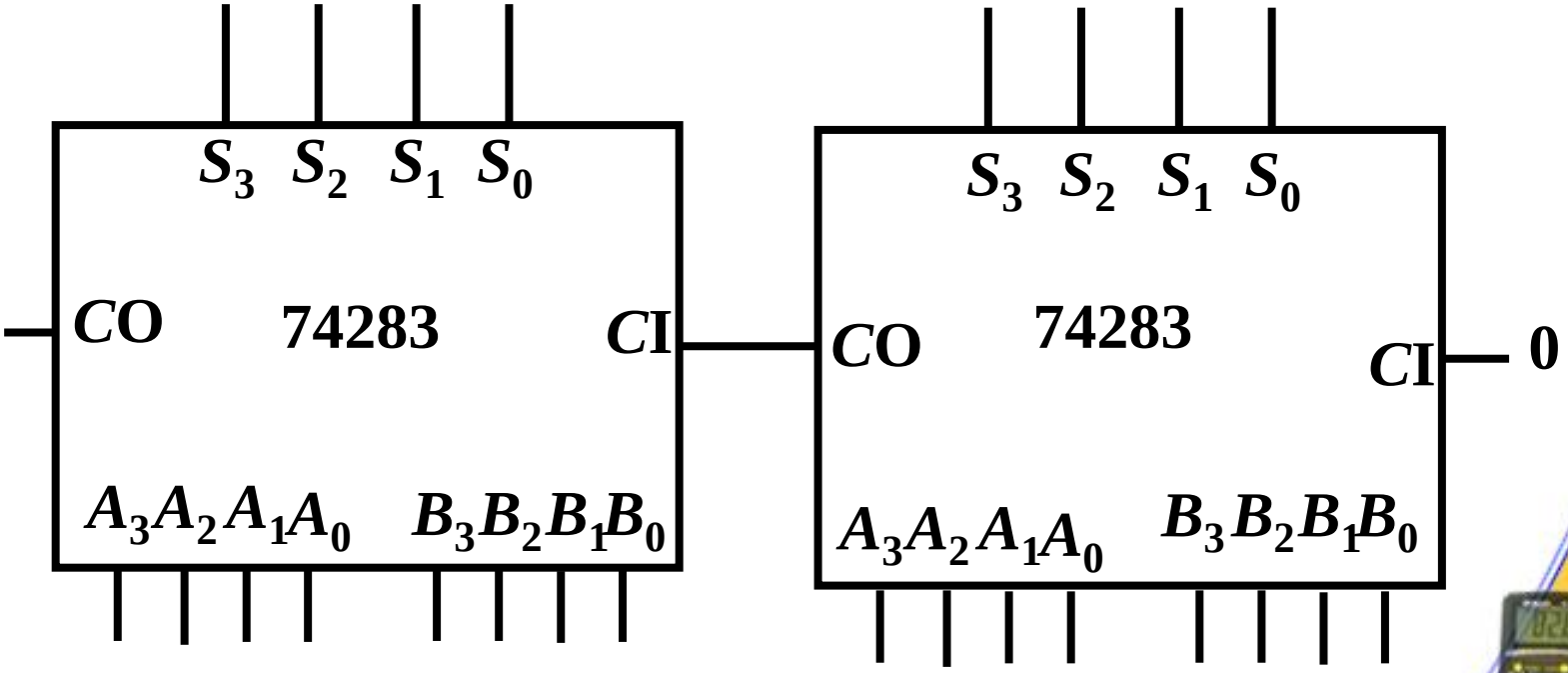


图6-27　由74LS283构成的代码转换电路

例: 两片74283构成八位二进制数加法器。

- 解：按照加法的规则，低四位的进位输出CO应接高四位的进位输入CI，而低四位的进位输入应接0。逻辑图如图所示。



两片74283构成八位二进制加法器

BCD码加法器

- BCD（Binary Coded Decimal）码是用二进制代码分别表示十进制数各位的代码组合。
- 由于每位十进制数最大为9，所以BCD码相加时，其值超过9的位必须通过减10或加6加以调整。
- 因此BCD码相加时，其和有需要调整和不需要调整两种情况：

一种和数为0-9，不需要调整；

另一种和数为10-18，需要调整

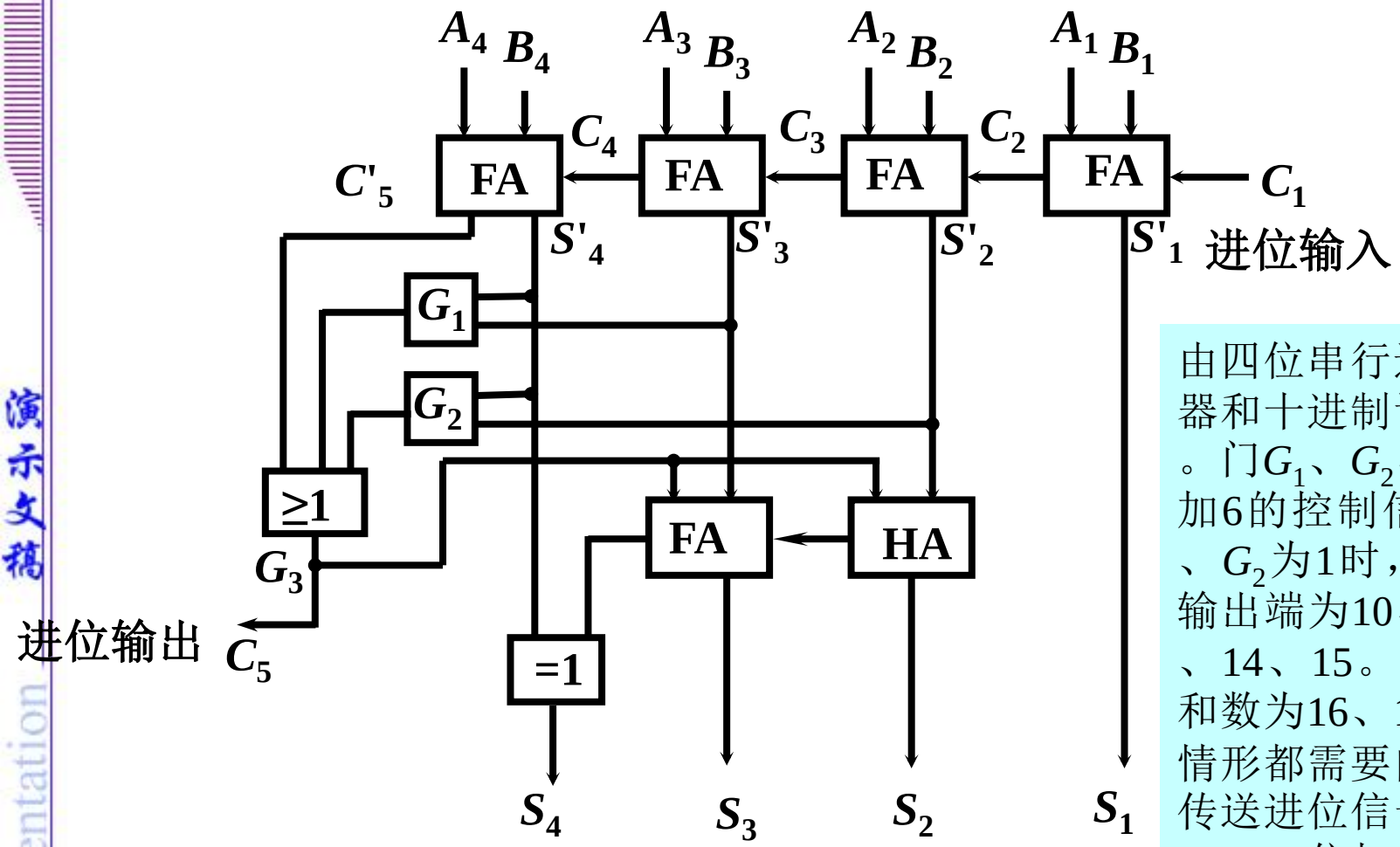
```
  0011
+）0100
------
  0111
```

```
  0111
+）0101
------
  1100
```

非BCD码
需要调整

```
  1100
+）0110
------
 1 0010
```

图为一位BCD码并行加法器逻辑图。



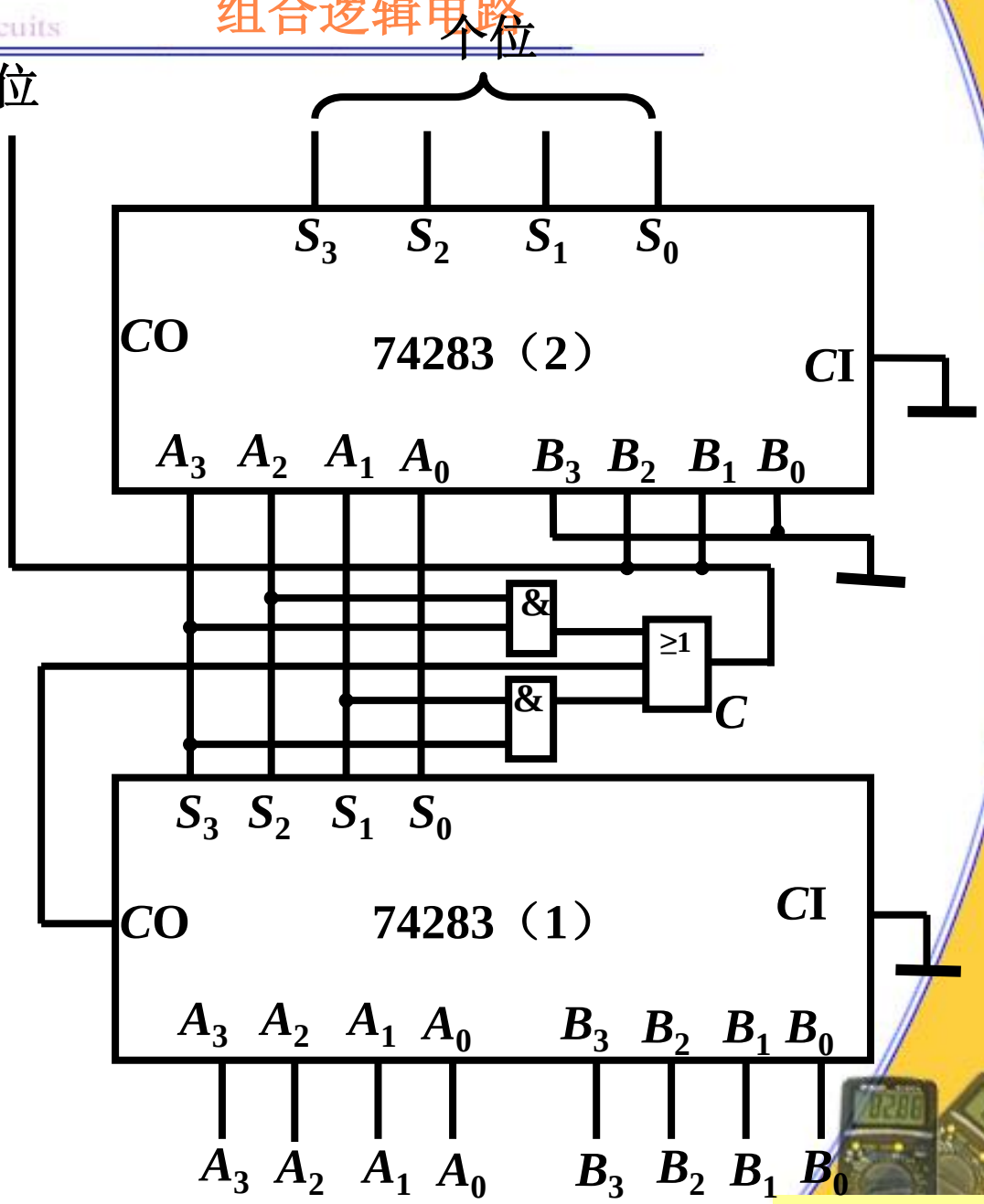
一位BCD码并行加法器

由四位串行进位并行加法器和十进制调整电路组成。门G_1、G_2、G_3用来产生加6的控制信号。当门G_1、G_2为1时，说明“和”输出端为10、11、12、13、14、15。当C'_5为1时，和数为16、17、18。以上情形都需要向高位加法器传送进位信号C_5，并对和S'_3、S'_2位加1，实现加6调整。

演示文稿 Presentation

S_3S_2 \ S_1S_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

$$C = S_3S_2 + S_3S_1$$



4.5 数值比较器

主要要求：

- 理解数值比较器的逻辑功能及应用。



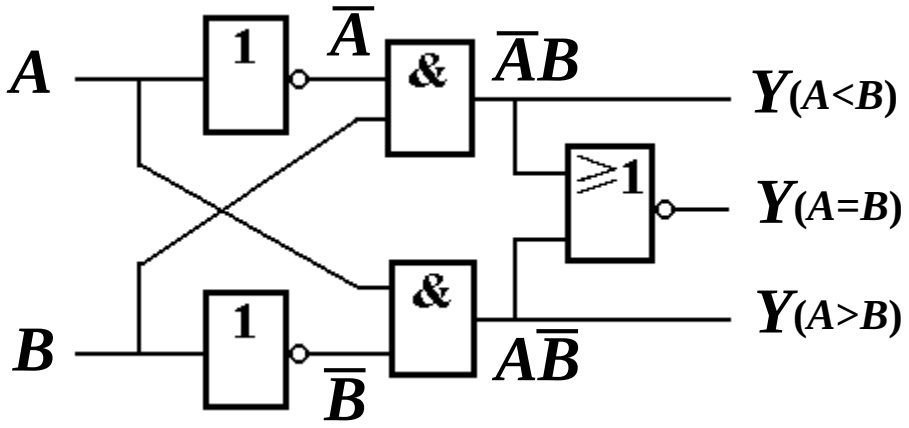
数值比较器

Digital Comparator，又称数字比较器。用于比较两个数的大小。

（一）1 位数值比较器

$$\begin{cases} Y(A>B)=A\overline{B} \\ Y(A<B)=\overline{A}B \\ Y(A=B)=\overline{A}\,\overline{B}+AB=\overline{A\oplus B} \end{cases}$$

输 入		输 出		
A	B	$Y_{(A>B)}$	$Y_{(A<B)}$	$Y_{(A=B)}$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1



（二）多位数值比较器

可利用 1 位数值比较器构成

比较原理：从最高位开始逐步向低位进行比较。

例如　比较 $A = A_3A_2A_1A_0$ 和 $B = B_3B_2B_1B_0$ 的大小：

若 $A_3 > B_3$，则 $A > B$；若 $A_3 < B_3$，则 $A < B$；若 $A_3 = B_3$，则需比较次高位。

若次高位 $A_2 > B_2$，则 $A > B$；若 $A_2 < B_2$，则 $A < B$；若 $A_2 = B_2$，则再去比较更低位。

依次类推，直至最低位比较结束。

四位数值比较器74LS85

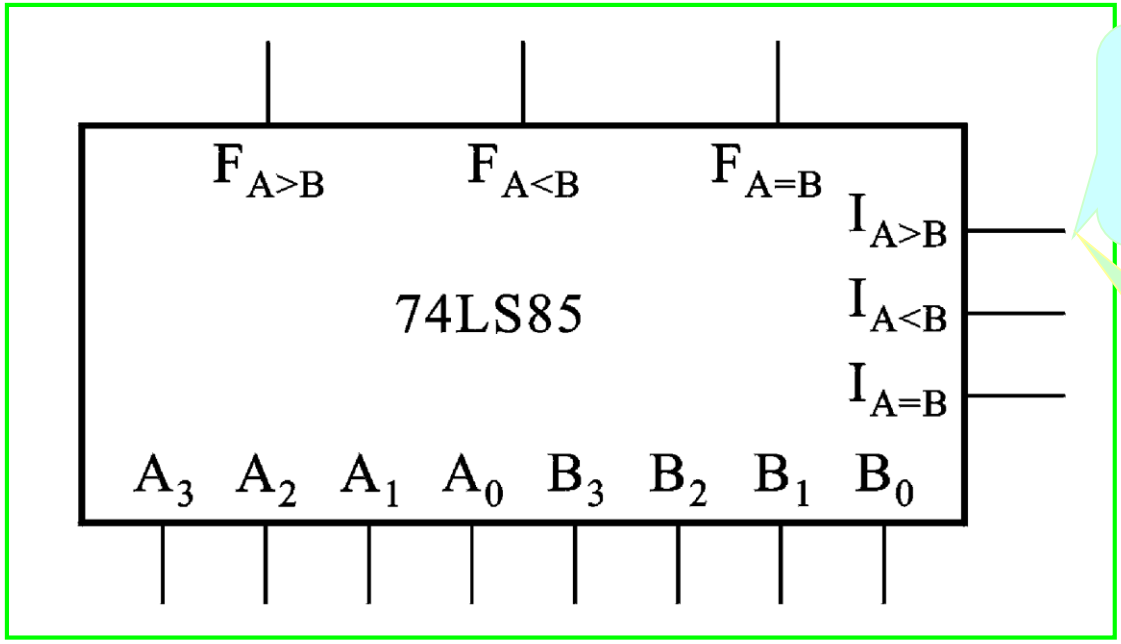
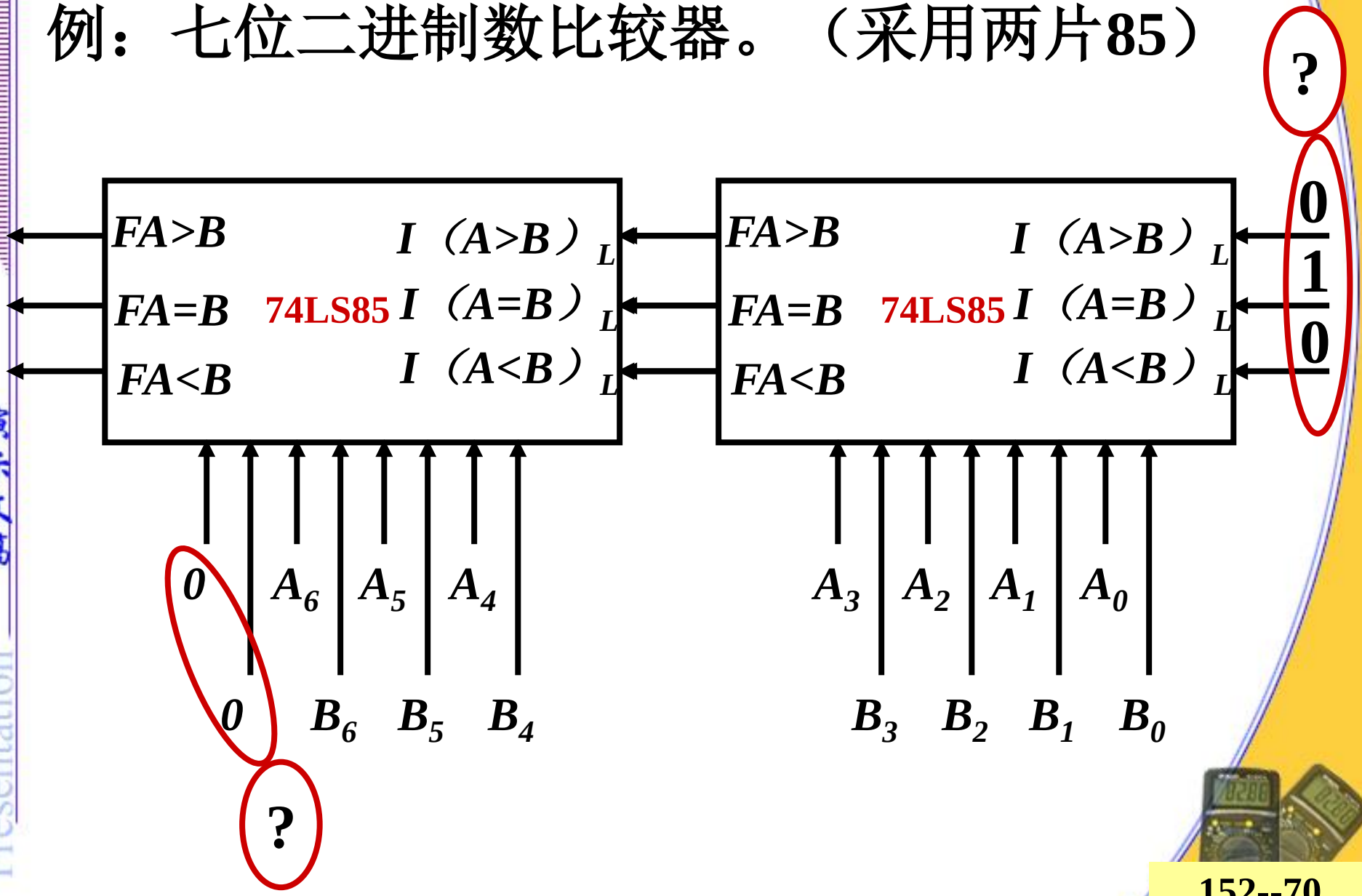


图6-28　74LS85的逻辑符号

74LS85的功能表

输入				级联输入			输出		
A_3, B_3	A_2, B_2	A_1, B_1	A_0, B_0	$I_{A>B}$	$I_{A<B}$	$I_{A=B}$	$F_{A>B}$	$F_{A<B}$	$F_{A=B}$
1 0	×	×	×	×	×	×	1	0	0
0 1	×	×	×	×	×	×	0	1	0
$A_3=B_3$	1 0	×	×	×	×	×	1	0	0
$A_3=B_3$	0 1	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	1 0	×	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	0 1	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	1 0	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	0 1	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	0	0	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	1	0	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	0	1	0	0	1

例：七位二进制数比较器。（采用两片85）



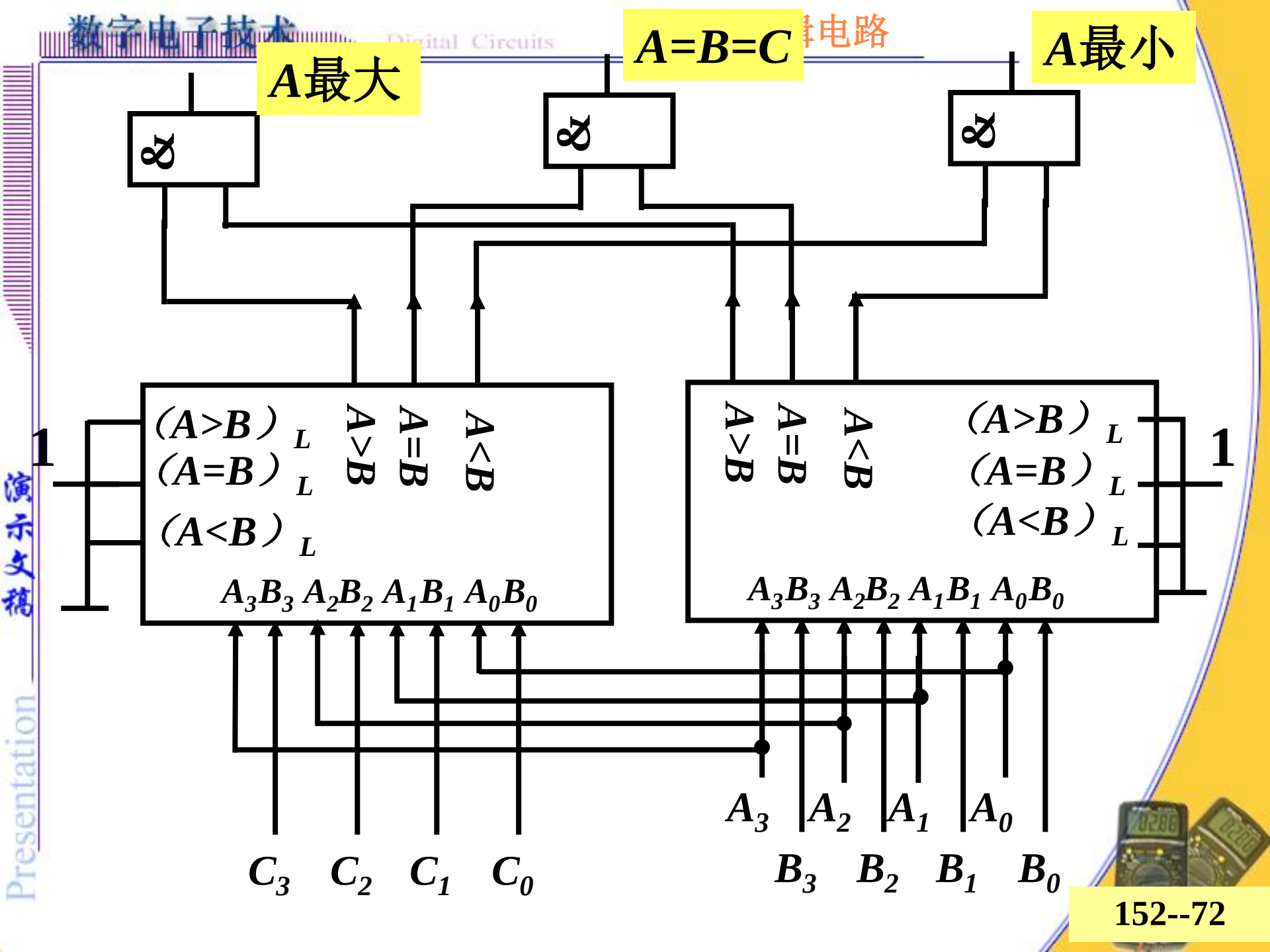
例：设计三个四位数的比较器，可以对*A*、*B*、*C*进行比较，能判断：（1）三个数是否相等。（2）若不相等，*A*数是最大还是最小。

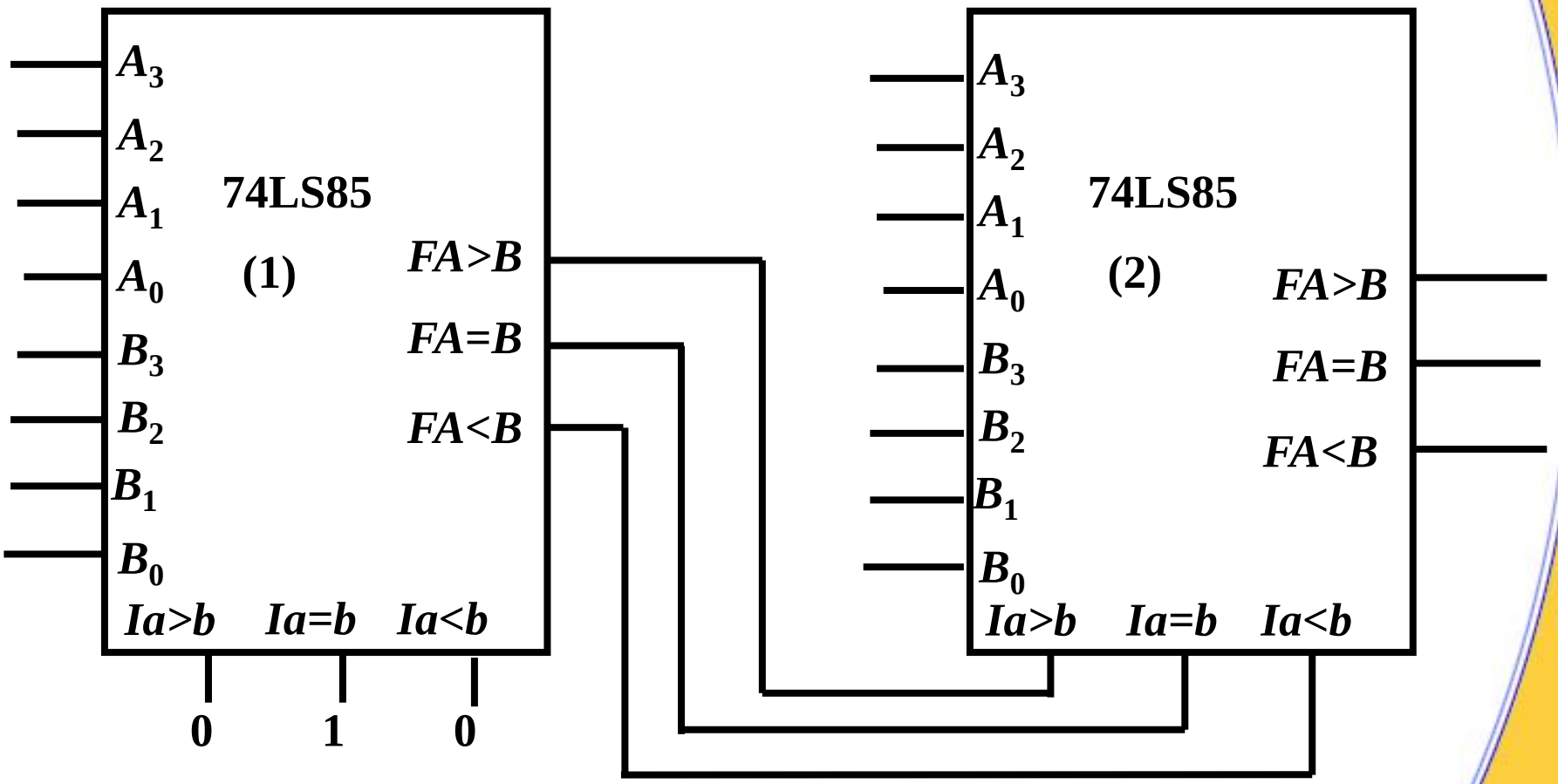
比较原则：

先将A与B比较，然后A与C比较，若*A*=*B* *A*=*C*，则*A*=*B*=*C*；若*A*>*B* *A*>*C*，则*A*最大；若*A*<*B* *A*<*C*，则*A*最小。

可以用两片74LS85实现。

演示文稿 Presentation


A最大
A=B=C
A最小
&
&
&
1
(A>B) L
(A=B) L
(A<B) L
A>B
A=B
A<B
A3B3 A2B2 A1B1 A0B0
A>B
A=B
A<B
(A>B) L
(A=B) L
(A<B) L
1
A3B3 A2B2 A1B1 A0B0
C3 C2 C1 C0
A3 A2 A1 A0
B3 B2 B1 B0



构成的八位数值比较的逻辑图

作业

雨课堂中的测试题5

演示文稿 Presentation

